

ETL & ELT Pipelines

From first principles to production architecture: extraction, transformation, loading, orchestration, quality, governance and real design practice.

ETL



Shape the data first, then load only curated results.

ELT



Land raw data first, then shape it inside the platform.

What you will be able to do

Explain

Define ETL and ELT precisely, trace where each came from, and describe how data travels from source to insight.

Decide

Choose ETL, ELT or a hybrid using explicit criteria: scale, latency, governance, skills and cost.

Build

Design idempotent extraction, testable transformations and safe loading with staging, merges and backfills.

Operate

Orchestrate, monitor and govern pipelines in production, with quality gates, lineage and incident response.

The test of mastery

By the end you can sketch a production-grade pipeline for a new domain on one page and defend every design choice.

The roadmap: thirteen parts

Depth increases steadily. Parts 1 to 4 build the mental model, 5 to 9 the engineering craft, 10 to 12 the production discipline.

Part 01 Foundations	Part 02 ETL in depth	Part 03 ELT in depth	Part 04 ETL vs ELT	Part 05 Data modelling
Part 06 Extraction deep dive	Part 07 Transformation deep dive	Part 08 Loading deep dive	Part 09 Orchestration	Part 10 Quality and governance
Part 11 Architecture patterns	Part 12 Design practice	Part 13 Reference and review	Levels 1 Foundation to 5 Production	Colour key Amber ETL · Teal ELT

A depth ladder: five levels of mastery

How to read the ladder

Each part revisits the same pipeline with more rigour. Vocabulary from earlier levels is assumed later, so a fast reader can skim Level 1 and a newcomer can stop after Level 3 with a solid working model.

LEVEL 1 Foundation

Parts 1–2

Vocabulary, history,
ETL mechanics

LEVEL 2 Core

Parts 3–4

ELT, tooling and the
trade-offs

LEVEL 3 Intermediate

Parts 5–6

Modelling and
extraction patterns

LEVEL 4 Advanced

Parts 7–9

Transform, load and
orchestrate at scale

LEVEL 5 Production

Parts 10–12

Quality, architecture,
real designs

REFERENCE Review

Part 13

Glossary, cheat
sheets, self-check

Conventions used throughout

Amber means ETL

Transform happens before load, in a dedicated engine or staging area.

Teal means ELT

Raw data loads first; transformation runs inside the target platform.

Slate means shared

Concepts common to both patterns: sources, modelling, orchestration, quality.

Code panels

Dark panels hold SQL, Python or YAML. Numbered notes beside them explain the lines that matter.

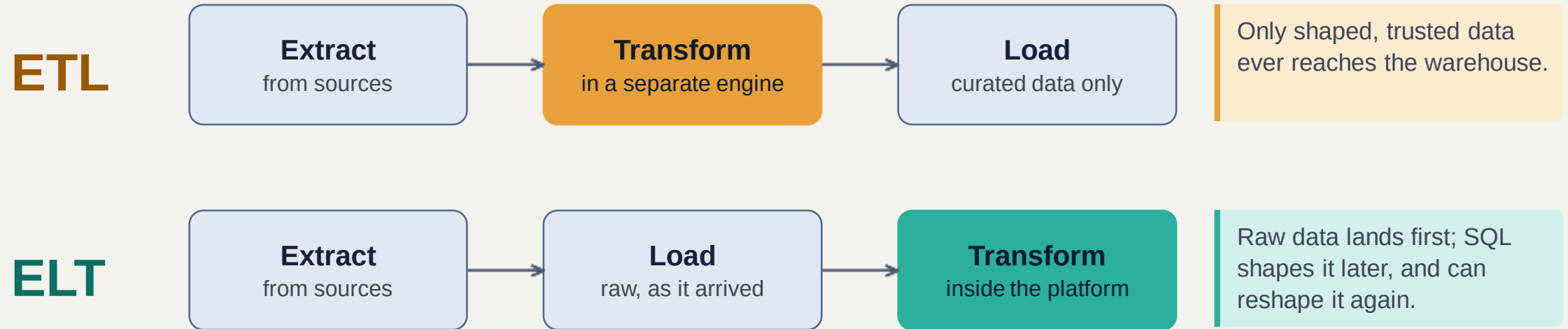
Illustrative charts

Charts marked illustrative show conceptual shapes, not benchmarks or measured results.

Prerequisites

Comfortable SQL, some Python, and a basic idea of relational tables and joins.

Hold this picture in your head



Same three verbs, different order. The order decides where compute runs, who owns the logic, and how easily results can be re-derived.

01

PART 01 · LEVEL 1 · FOUNDATION

Foundations: why data has to move

Before comparing ETL and ELT, understand the problem they solve: getting data out of many systems and into a form people can trust.

The data problem

Pipeline anatomy

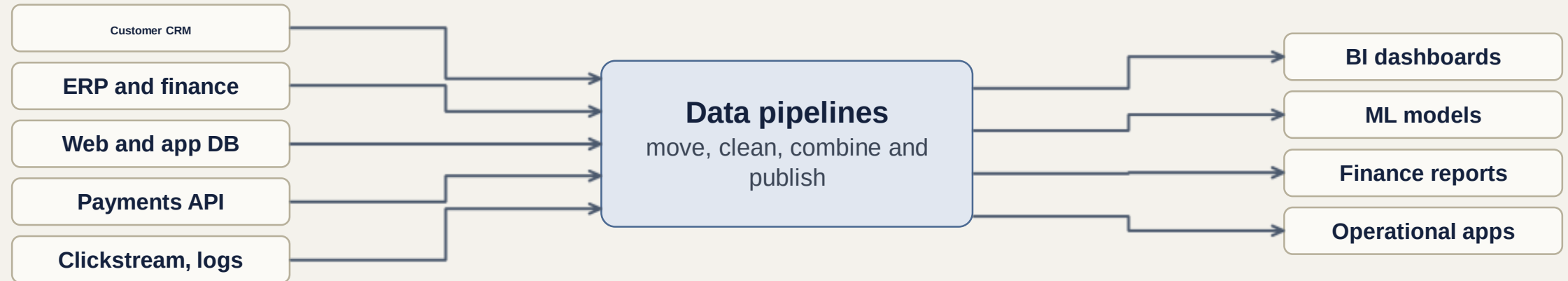
Sources and destinations

History

Batch and streaming

The problem: valuable data lives everywhere

SOURCES



Inconsistent definitions

'Customer' and 'revenue' mean different things in each system.

Different formats and clocks

Time zones, encodings, keys and update cadences rarely agree.

No single trusted copy

Analysts reconcile numbers by hand, and nobody can prove which one is right.

Anatomy of a data pipeline

ETL and ELT differ here



Where data comes from: a taxonomy of sources

Source type	Typical examples	How data is read	Main risk
OLTP databases	PostgreSQL, MySQL, Oracle	Queries, replicas, CDC	Load on production systems
SaaS applications	CRM, billing, support tools	REST APIs, vendor connectors	Rate limits, silent schema change
Files, object storage	CSV, JSON, Parquet, Avro	Scheduled or event-driven pickup	Late, partial or malformed files
Event streams	Kafka, Kinesis, Pub/Sub	Consumer groups with offsets	Duplicates, out-of-order events
Logs and telemetry	App logs, metrics, clickstream	Agents, collectors, batch drops	Huge volume, loose structure
Third-party data	Market, open, partner feeds	SFTP, APIs, downloads	Licensing, unfixable quality
Legacy systems	Mainframes, flat-file exports	Nightly exports, fixed-width	Undocumented, brittle formats

Where data lands: a taxonomy of destinations

Data warehouse

Structured, governed, SQL-first store tuned for analytics. Schema-on-write with predictable performance.

Data lake

Cheap object storage for raw and semi-structured files. Flexible, but easy to turn into a swamp.

Lakehouse

Open table formats on object storage add transactions and SQL to lake data in one platform.

Data mart

A subject-focused slice, such as finance or sales, modelled for one team and kept simple.

Feature store

Serves consistent features for training and low-latency inference with point-in-time correctness.

Serving layer

Caches, search indexes and APIs that put curated data back into applications.

The family tree: pipelines, ETL and ELT

Data pipelines: any automated movement and preparation of data

ETL

Transform in an engine, then load curated data

ELT

Load raw data, transform inside the target

Replication and CDC

Mirror changes with little or no transformation

Streaming pipelines

Continuous processing of events in motion

Reverse ETL

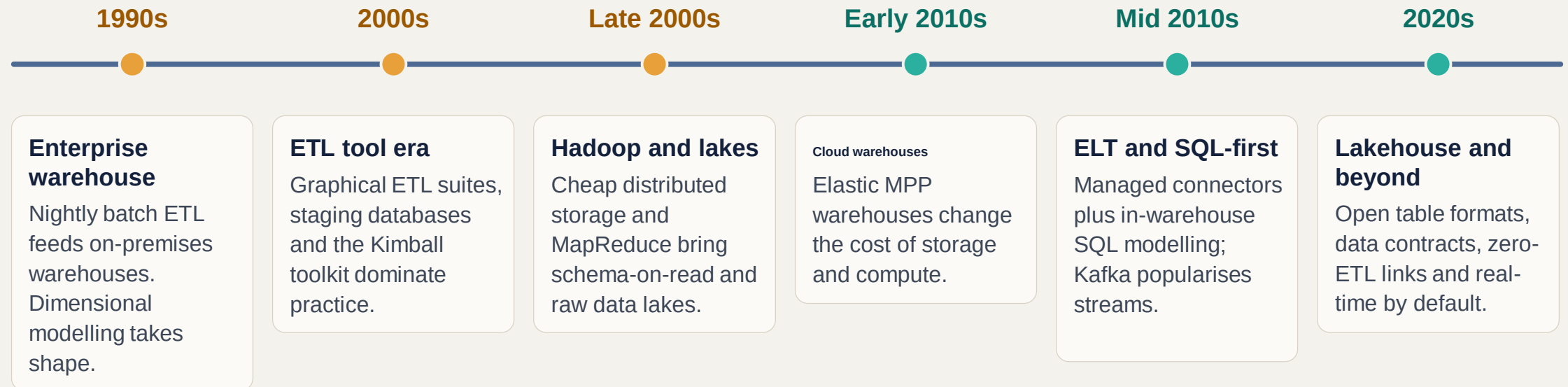
Push modelled warehouse data back into apps

Read the hierarchy

ETL and ELT are integration patterns. Every ETL or ELT job is a pipeline, but not every pipeline is ETL or ELT.

ETL and ELT only say where transformation happens relative to loading. Everything else about pipelines is orthogonal.

A short history of moving data



Approximate eras. Adoption overlapped and varies by industry. Amber leans ETL, teal leans ELT.

Batch, micro-batch and streaming

Dimension	Batch	Micro-batch	Streaming
Latency	Hours to days	Seconds to minutes	Milliseconds to seconds
Unit of work	Bounded dataset	Small bounded time slices	Individual events, unbounded
Complexity	Lowest	Moderate	Highest: state, ordering, replay
Cost profile	Cheap, bursty compute	Steady, moderate compute	Always-on compute
Typical tooling	SQL, Spark batch, Airflow	Spark Structured Streaming	Kafka, Flink, Kinesis
Best for	Reporting, finance close	Near-real-time dashboards	Fraud, alerting, personalisation

The latency spectrum

Cheaper and simpler to run



Faster and more complex to run

Days

Hours

Minutes

Seconds

Sub-second

Compliance and close

Regulatory reporting, month-end finance close, archival loads.

Classic ETL window

Nightly ETL, daily dashboards, overnight model refresh.

Hourly ELT

Hourly and near-real-time BI, micro-batch ELT runs.

Operational

Ops dashboards, alert enrichment, CDC replication.

Decisioning

Fraud scoring, recommendations, IoT control loops.

Choose the slowest latency the business can tolerate. Every step toward the right multiplies operational cost and failure modes.

02

PART 02 · LEVEL 1 · FOUNDATION

ETL in depth: transform, then load

The classical pattern. Extract from sources, reshape the data in a dedicated engine, and load only curated results into the warehouse.

Definition

Architecture

Extract

Transform

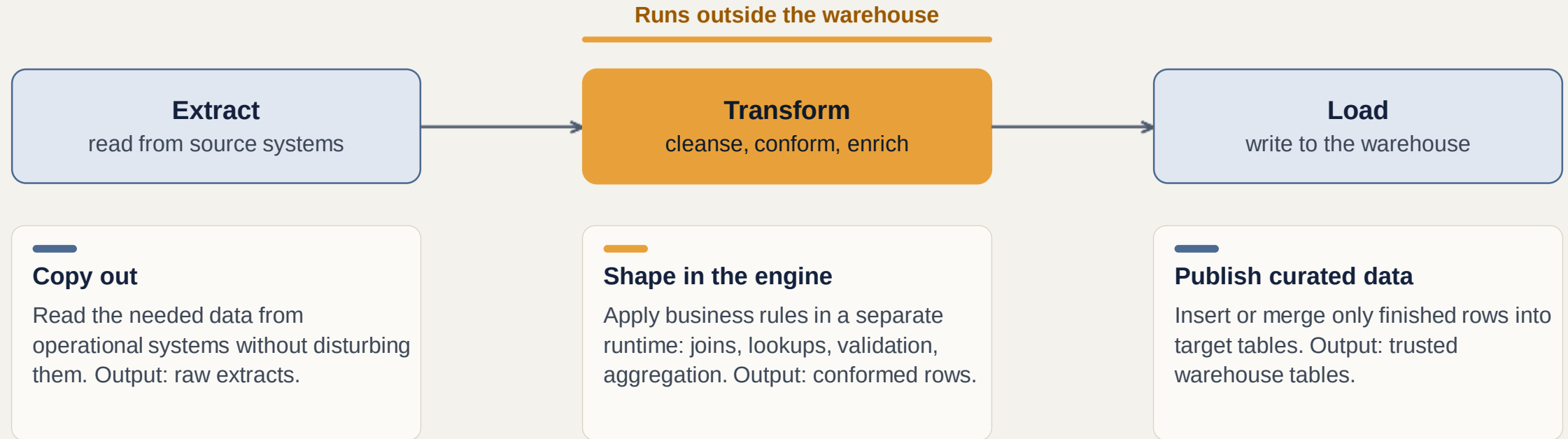
Staging

Load order

Tools

Performance

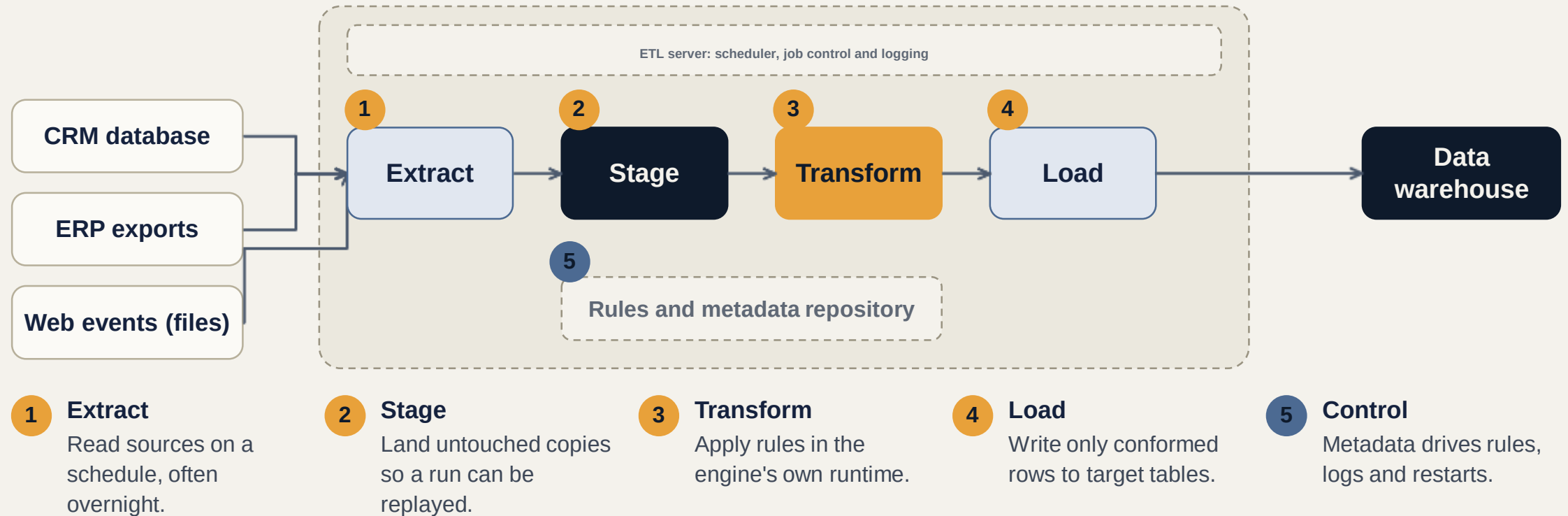
ETL defined: three stages, one direction



Working definition

ETL is a pattern, not a product: any process that transforms data before loading it into the target follows it.

The ETL architecture: engine, staging, target



The extract step in ETL

What extraction must do

- Read only what the run needs
- Protect source performance
- Preserve data exactly as read
- Fail loudly on partial reads
- Record what was read and when

Metadata captured per run

- Batch identifier and run start time
- Source system and object name
- Row counts read and rejected
- High-watermark or offset reached
- Checksum or file manifest entry

JDBC and ODBC

Query relational sources directly or via replicas.

APIs

REST or SOAP calls with paging and rate limits.

File transfer

SFTP drops, exports and object storage.

Queues

Consume messages with tracked offsets.

The transform step: seven families of work

Transformation	What it does	Example
Cleanse	Fix or remove bad values	Trim spaces, replace 'N/A' with NULL
Standardise	Force one format and unit	2024/3/7 becomes 2024-03-07
Deduplicate	Keep one row per real entity	Latest record per customer_id
Conform	Align keys and codes across systems	CRM 'UK' and ERP 'GB' become 'GB'
Enrich	Add data from reference sources	Attach region from a postcode lookup
Aggregate	Roll detail up to reporting grain	Sum order lines to daily revenue
Derive	Compute new business fields	margin = revenue minus cost

Most pipeline defects and most business value both live in this step. Treat transformation rules as tested, versioned code.

A worked example: one order, three shapes

1 Source row

customer: 'ACME Corp.'

ccy: GBP
amount: '1,250.00'
status: shp

2 Staging copy

customer: 'ACME Corp.'

ordered: 2024/3/7
_batch_id: 20240307T0200
amount: '1,250.00'

3 Warehouse row

order_date: 2024-03-07

customer_key: 5071
amount_gbp: 1250.00
status: SHIPPED

Messy: padded text, slash dates, a thousands separator and a cryptic status code.

Untouched, plus batch metadata, so the run can be replayed without re-reading the source.

Typed, conformed and keyed with surrogate keys. This is the only shape analysts ever see.

The transformation ran in the ETL engine. The warehouse never receives the messy form.

Staging areas: the ETL safety buffer

Why ETL stages data

- Decouples source uptime from transform timing
- Lets a failed run restart without re-extracting
- Reads production once, not repeatedly
- Enables set-based joins across sources
- Keeps an audit copy of what arrived

Staging design rules

- Keep business rules out of staging tables
- Stamp every row with batch id and load time
- Restrict access: staging may hold raw PII
- Set a retention period after successful load
- Never let reports query staging directly

Transient

Truncated at the start of each run. Smallest footprint, no history.

Persistent

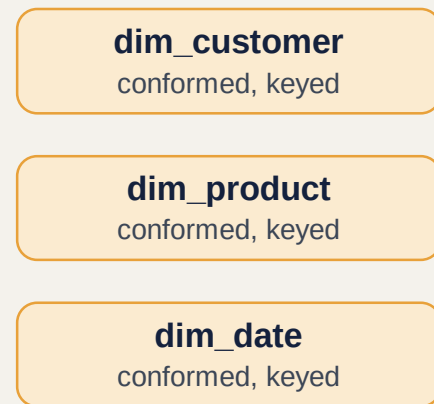
Retained for a window of batches. Enables replay and comparison.

Archive

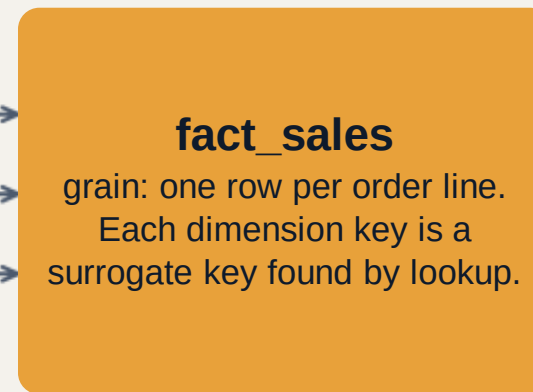
Immutable files in cheap storage. Audit trail and disaster recovery.

The load step: dependency order matters

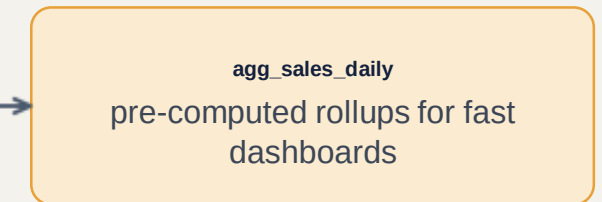
1 Load dimensions first



2 Look up keys, load facts



3 Refresh aggregates



- 1 Dimensions before facts**
A fact row needs the surrogate key of every dimension it references.

- 2 Unknown members**
Late-arriving keys map to an 'Unknown' row instead of failing the load.

- 3 Aggregates last**
Rollups rebuild from committed facts, never from half-loaded data.

Classical ETL tools and their heritage

Category	Examples	How you build	Typical strength
Graphical ETL suites	Informatica PowerCenter, IBM DataStage, Talend	Drag-and-drop mappings	Governed, audited enterprise flows
Microsoft ecosystem	SQL Server Integration Services (SSIS)	Visual packages and T-SQL	Tight SQL Server integration
Open-source visual	Pentaho Data Integration, Apache NiFi	Flow canvases	Low licence cost, flexible routing
Cloud-managed ETL	AWS Glue, Azure Data Factory, Google Cloud Dataflow	Managed jobs and pipelines	No servers to run, elastic scale
Distributed engines	Apache Spark, Apache Beam, Apache Flink	Code in Scala, Python, SQL	Very large volumes, custom logic
Scripted Python	pandas, Polars, custom scripts	Code in repositories	Small data, full control, low cost

Product names are examples of each category, not recommendations. Capabilities change quickly; verify against current documentation.

Where ETL wins and where it struggles

Where ETL wins

- Sensitive data masked before it lands
- A small, curated warehouse keeps storage cost low
- Complex non-SQL logic runs in a suitable engine
- Strong fit for on-premises and legacy targets
- Mature tooling, audit trails and governance

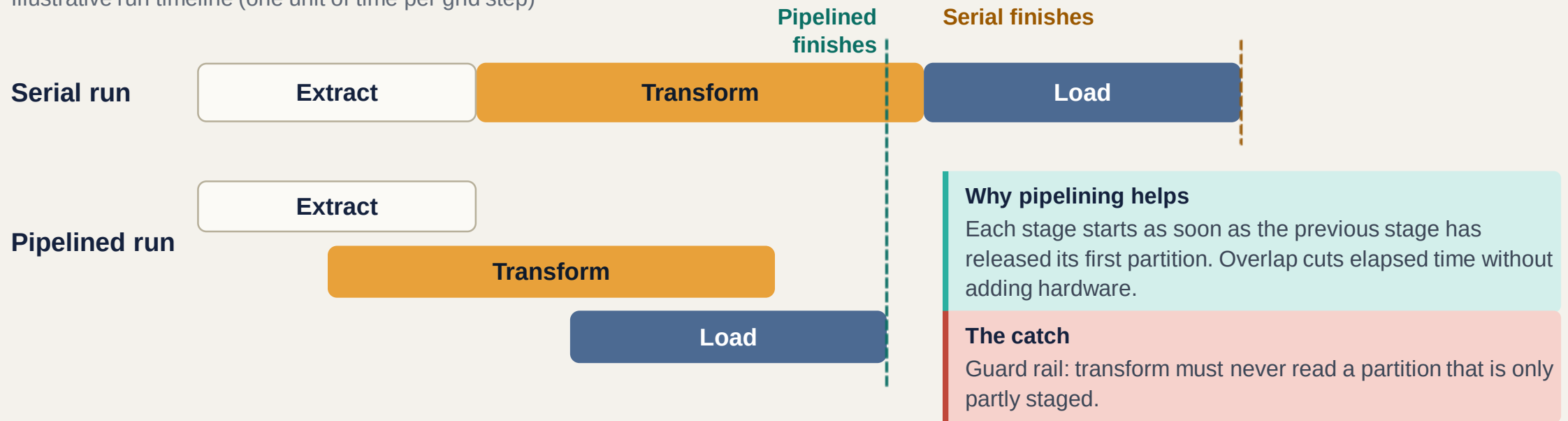
Where ETL struggles

- The transform engine becomes the scaling bottleneck
- Changing a rule often means re-extracting sources
- Raw history is lost unless you archive it
- Long change cycles for every new report field
- Specialist skills and proprietary tool lock-in

ETL is not obsolete. It is the right answer when data must be shaped or protected before it lands.

Performance: where ETL run time goes

Illustrative run timeline (one unit of time per grid step)



Elapsed time falls when stages overlap and when each stage works on partitions in parallel.

ELT in depth: load first, transform in place

The modern pattern. Copy raw data into a scalable platform, then use its own compute and SQL to shape it, again and again if needed.

Definition

Why it emerged

Architecture

Raw layer

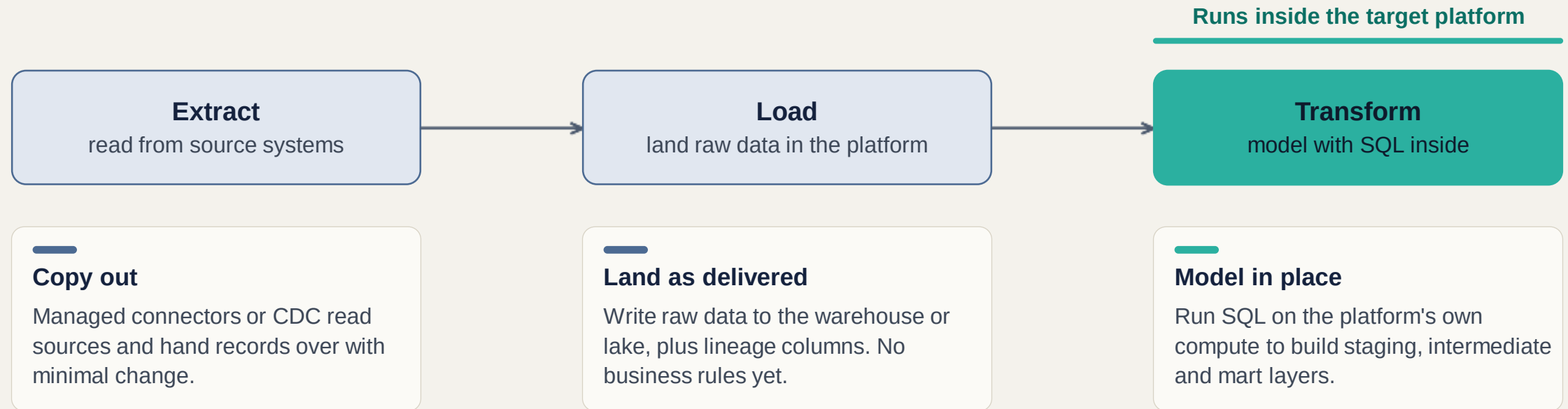
SQL models

Layers

Decoupled compute

Toolchain

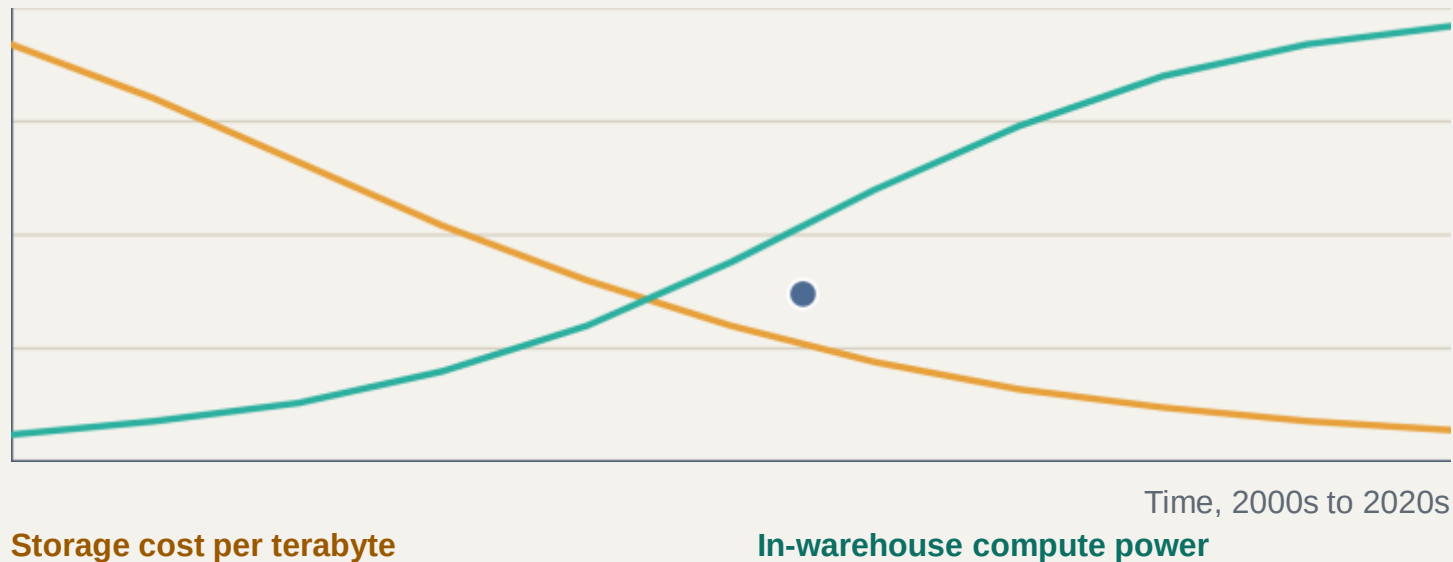
ELT defined: land first, transform in place



Working definition

ELT keeps a faithful raw copy, so every transformation can be revised and replayed without touching the source again.

Why ELT emerged: three enabling shifts



Illustrative shapes to show direction of change. They are not measured data.

1 Scalable SQL engines

Columnar, massively parallel warehouses run big SQL transforms fast.

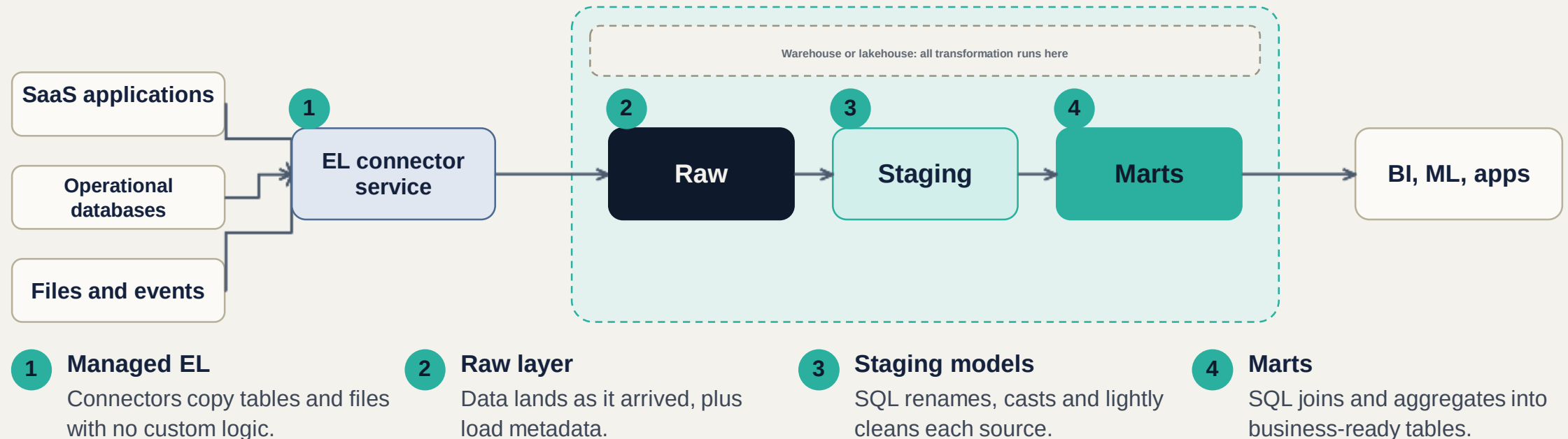
2 Cheap, elastic storage

Cheap storage separated from compute makes keeping raw data affordable.

3 Tooling maturity

Managed connectors and SQL modelling tools removed most custom code.

The ELT architecture: connectors, raw, models



The raw layer: an immutable landing zone

```
raw.shop_orders

create table raw.shop_orders (
  id bigint,
  payload json, -- [1]
  _source varchar, -- [2]
  _batch_id varchar,
  _loaded_at timestamp -- [3]
);

-- [4] append only, never update raw
insert into raw.shop_orders
select id, payload, 'shop_db',
       :batch_id, current_timestamp
from landing.orders_20240307;
```

- 1 Keep the payload whole**
Store the record as delivered, so later fixes need no re-extract.
- 2 Lineage columns**
Source and batch id make every row traceable to one load.
- 3 Load timestamp**
Finds the latest version and audits late arrivals.
- 4 Append only**
Raw is history. Corrections happen downstream in models.

Transformation as SQL models

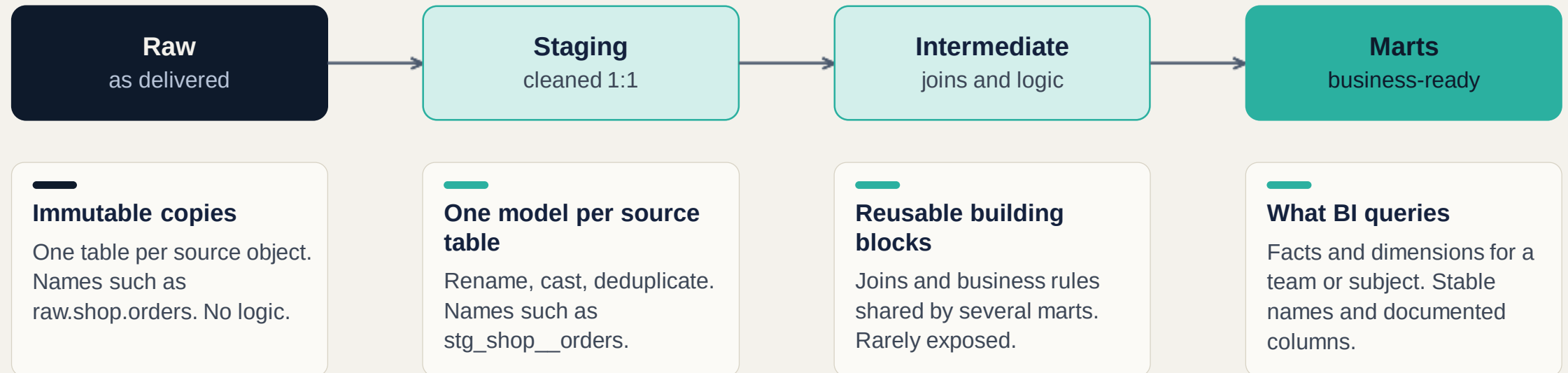
```
models/staging/stg_orders.sql
```

```
select
  id as order_id, -- [1]
  cast(customer_id as bigint) as customer_id,
  cast(ordered_at as timestamp) as ordered_at, -- [2]
  amount_cents / 100.0 as amount,
  upper(trim(status)) as status -- [3]
from raw.shop_orders
where id is not null -- [4]
```

- 1 Rename to business terms**
One consistent vocabulary from the first model onward.
- 2 Cast to real types**
Raw is often text. Models give every column a proper type.
- 3 Normalise codes**
Trim and case-fold so joins and filters behave.
- 4 Filter, do not judge**
Drop only rows that are structurally unusable; test the rest.

Each model is a SELECT statement. The tool wraps it in the DDL that creates a view or table, and orders builds by dependency.

Layered modelling: raw, staging, intermediate, marts



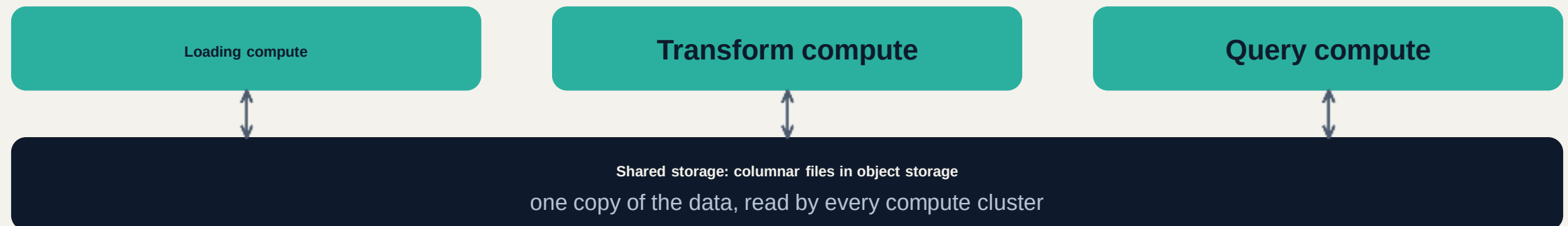
Each layer has one job. That discipline keeps SQL projects debuggable as they grow.

Compute inside the platform: decoupled from storage

Ingestion jobs scale up for big syncs, then release capacity.

Heavy model builds get their own compute and never slow dashboards.

Analyst queries scale with concurrency, isolated from batch work.



Because compute is separate and elastic, ELT pays for transformation only while it runs.

The modern ELT toolchain



Product names are examples of each layer, not recommendations. Any layer can be swapped without changing the pattern.

Where ELT wins and where it struggles

Where ELT wins

- Raw history kept, so logic can be re-run any time
- Warehouse compute scales for big transformations
- SQL skills reach analysts as well as engineers
- New sources arrive fast through managed connectors
- Fewer moving parts to operate

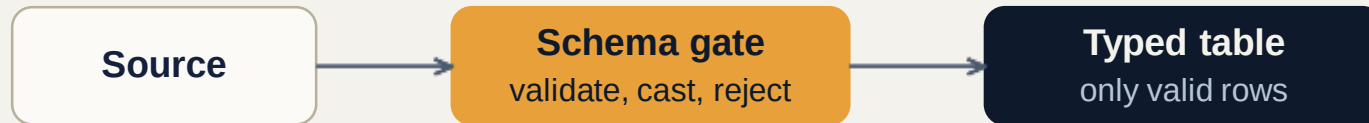
Where ELT struggles

- Raw PII lands in the warehouse and needs protection
- Compute bills can grow silently without controls
- Raw layers become swamps without conventions
- SQL is awkward for parsing, ML and custom code
- Quality issues surface late unless tests gate models

Every ELT weakness is manageable: masking policies, cost guardrails, naming standards and tests are part of the design.

Schema-on-write versus schema-on-read

Schema-on-write (classic ETL)



On schema change

A new source column is ignored, or breaks the load, until the mapping is changed.

Schema-on-read (ELT and lakes)



On schema change

The new column appears in raw at once. Models adopt it when ready, and nothing breaks.

ELT defers interpretation, which is flexible, but types and rules must still be enforced before data is trusted.

ETL versus ELT: choosing with evidence

Neither pattern wins in general. This part gives you criteria, trade-offs and a decision path you can defend to an architecture review.

Core differences

Data path

Cost

Governance

Reprocessing

Scorecard

Decision tree

Hybrid

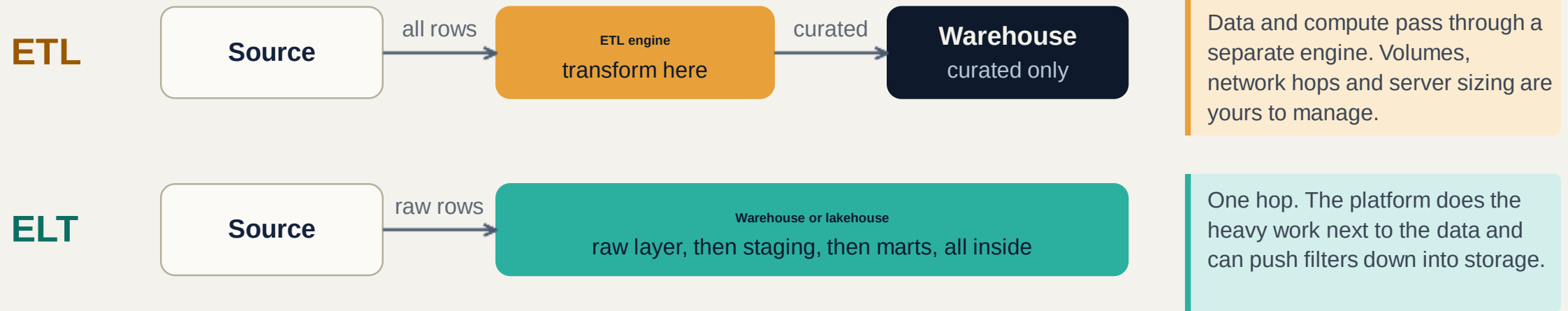
Myths

Side by side: the core differences

Dimension	ETL	ELT
Order of steps	Extract, transform, load	Extract, load, transform
Where transformation runs	A separate ETL engine	Inside the target platform
What the target holds	Curated, conformed data	Raw data plus modelled layers
Raw history	Lost unless archived	Kept by design
Changing a business rule	Often means re-extracting	Re-run SQL on retained raw
Typical skills	ETL tools, Java, Python, Spark	SQL, dbt, warehouse administration
Cost shape	Fixed servers and licences	Usage-based compute and storage
Best fit	Masking, legacy targets, non-SQL logic	Analytics on scalable platforms

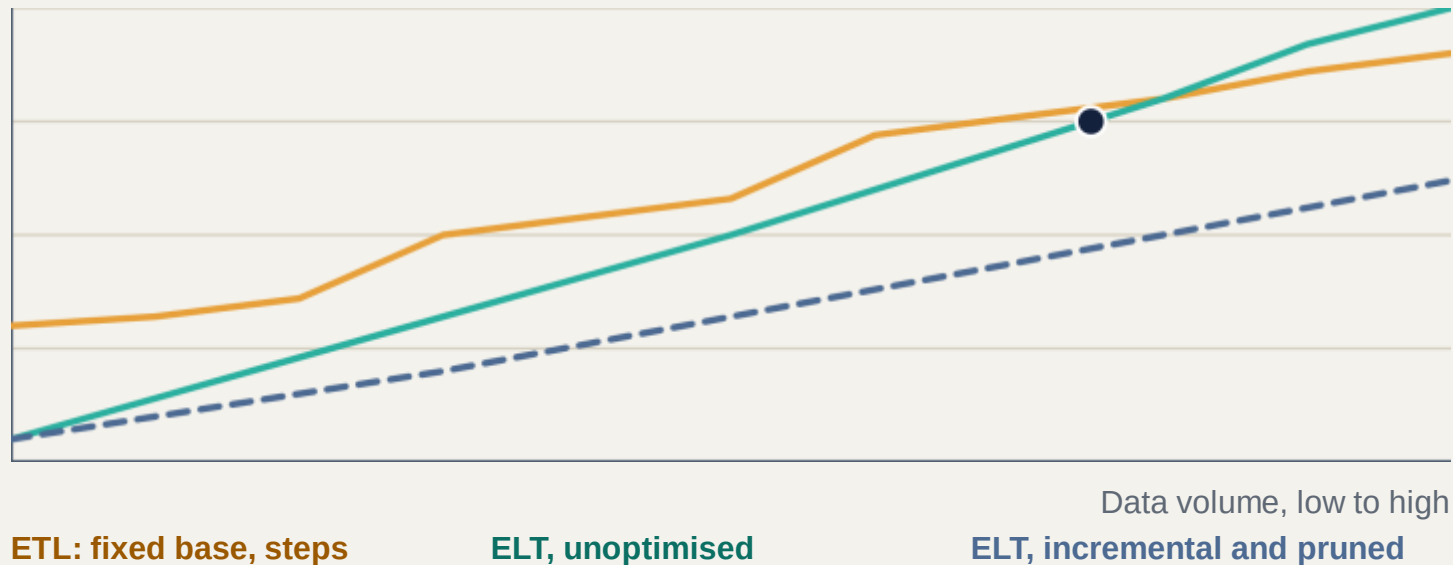
Read the table as tendencies. Real pipelines mix both, and the sections that follow show how to decide.

Where the data and the compute travel



The pattern decides who pays for transformation compute: your own servers, or the platform's elastic pool.

Cost model: fixed capacity against usage



Illustrative shapes only. Real cost depends on vendor pricing, workload and design.

Small volumes

ELT starts cheap because nothing is provisioned. ETL carries server and licence cost from day one.

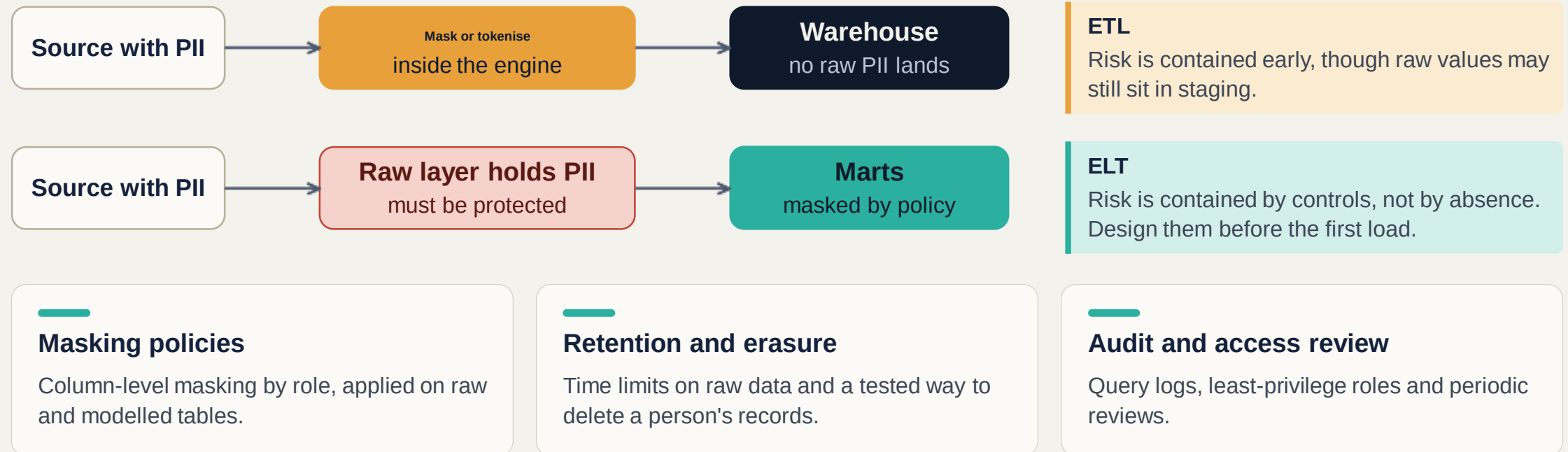
Crossover

Full rebuilds on every run make compute grow with data and can overtake fixed ETL cost.

Optimisation

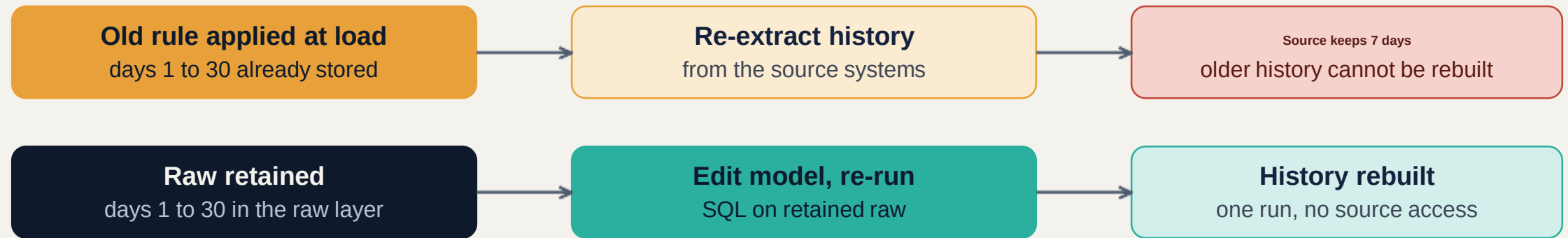
Incremental models, partition pruning and right-sized compute bend the ELT line down.

Governance and privacy: where sensitive data lands



Flexibility: reprocessing when a rule changes

Scenario: on day 30, the definition of 'active customer' changes



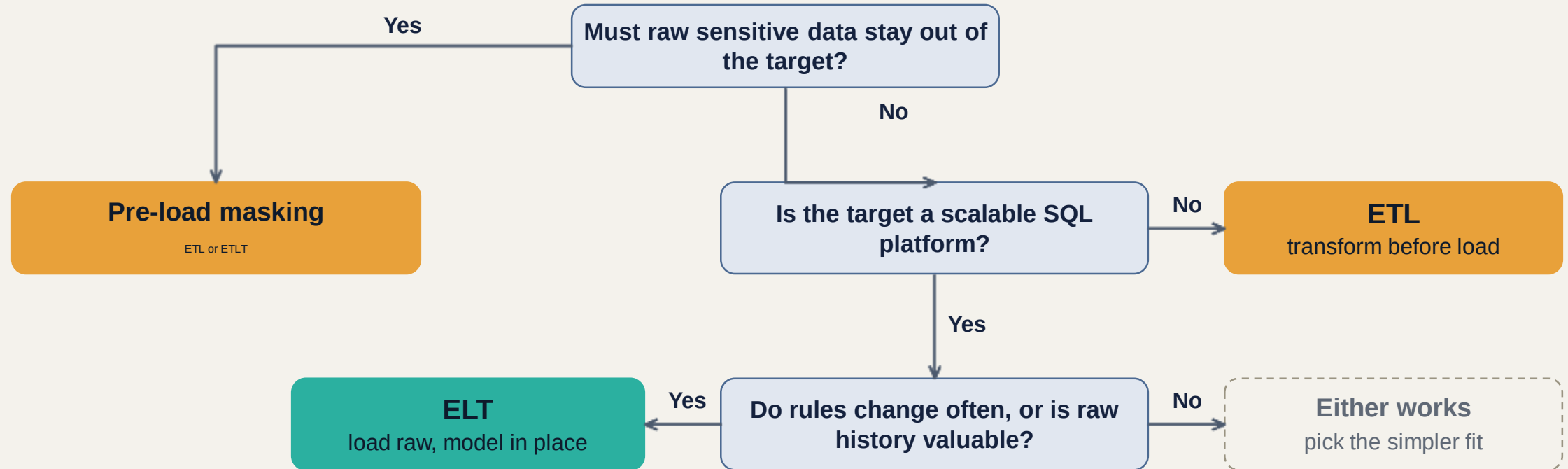
Reprocessing power is the strongest practical argument for keeping raw data. ETL teams recover it by archiving extracts.

A criteria scorecard

Which way does each constraint lean?

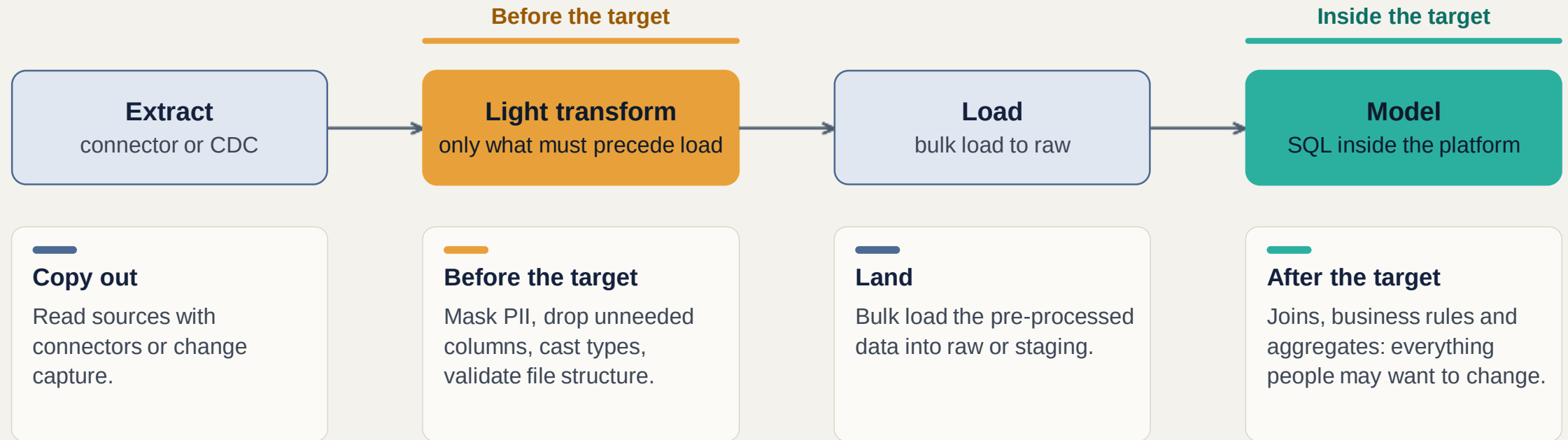
Raw sensitive data must not land	ETL	Mask or drop before the target sees it
Legacy or on-premises target	ETL	The target cannot run heavy SQL
Elastic cloud warehouse	ELT	Compute scales on demand
Business rules change often	ELT	Re-run models on retained raw
Analysts own transformations	ELT	SQL reaches more of the team
Complex non-SQL logic	ETL	A code engine is a better fit
High-volume event streams	Either	Depends on stream tooling and latency
Tiny data and few sources	Either	Simplicity matters most

A decision tree for the common cases



A starting heuristic. Confirm with cost modelling, team skills and a compliance review.

Hybrid: ETLT and selective pre-processing



ETLT: do the minimum before loading, and everything else after. Push work earlier only for privacy, format or cost reasons.

Six common myths



ELT means no transformation before load

Light steps such as decompression, type-casting and masking often still come first.



ETL is dead

It stays right for masking, legacy targets and non-SQL logic.



ELT is always cheaper

Cheap storage helps, but unmanaged compute and full rebuilds can cost more.



Raw data means a data lake

Raw is a layer, not a platform. Warehouses hold raw layers too.



SQL-only means simple

Large SQL projects need modularity, tests and review, just like code.



The tool decides the pattern

Constraints decide it. The same tool can often do either.

PART 05 · LEVEL 3 · INTERMEDIATE

Data modelling: shaping data to be used

A pipeline is only as useful as the model it produces. Learn the dimensional toolkit that makes warehouse data fast, consistent and explainable.

Analytical shape

Star schema

Grain

Fact types

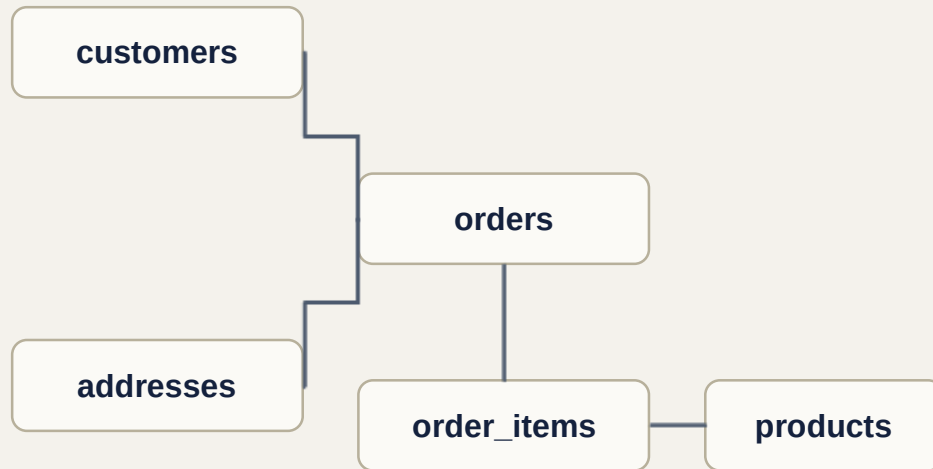
Dimensions and keys

SCD types

Modelling styles

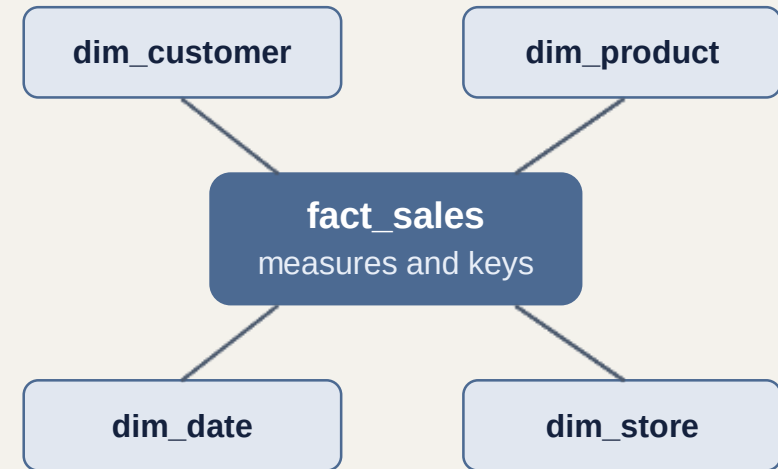
From operational shape to analytical shape

Operational: normalised for writes



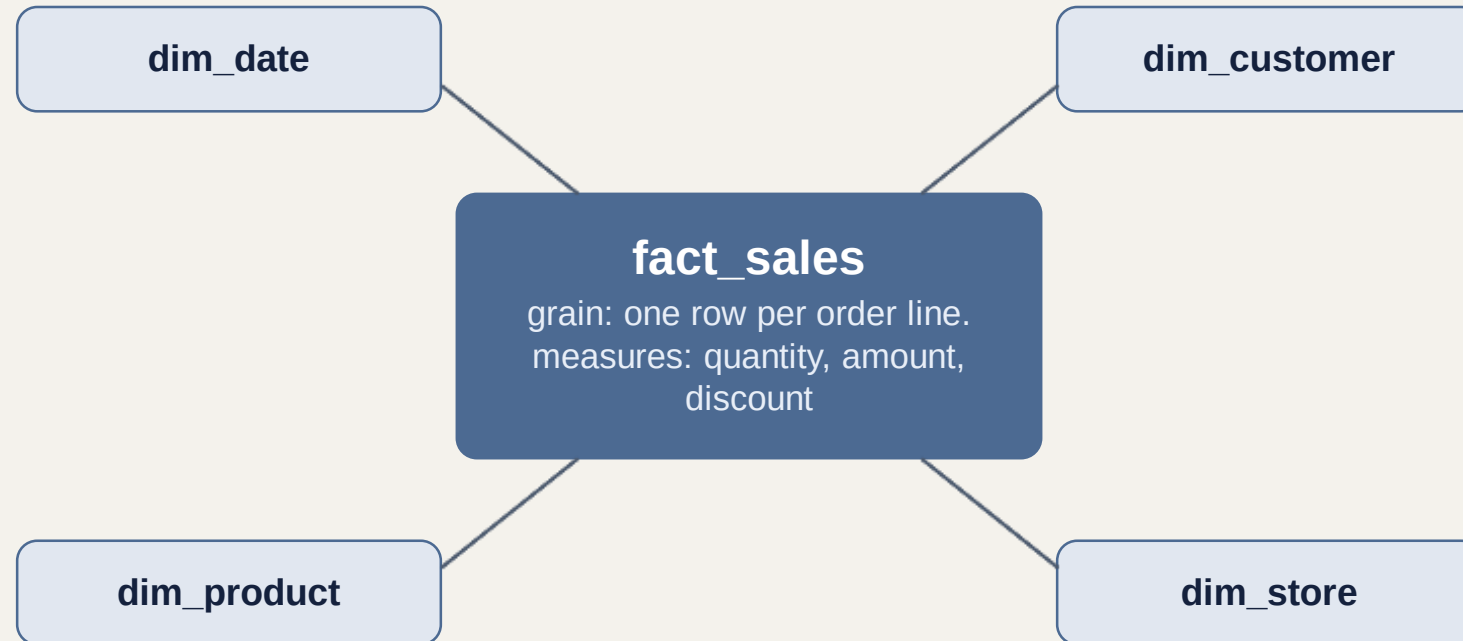
model for analysis
→

Warehouse: dimensional for reads



Operational schemas avoid redundancy so writes stay safe. Analytical schemas add redundancy on purpose so reads stay fast and simple.

Star schema anatomy



1 Grain

What one fact row means. Decide it first, in words, before choosing columns.

2 Measures

Numeric values you sum, average or count. Additive measures are the easiest to use.

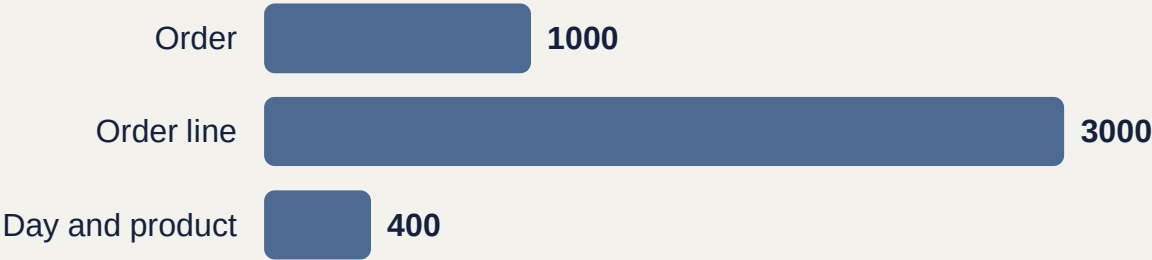
3 Keys

Foreign keys to dimensions are surrogate keys, so source id changes never break history.

Facts hold the numbers. Dimensions hold the context you filter and group by.

Grain: the most important decision

Grain	One row per	Can answer	Cannot answer
Order	order	Revenue by customer	Revenue by product
Order line	product on an order	Revenue by product and customer	Behaviour before purchase
Day and product	product per day	Daily sales trends	Individual orders



Finer grain, more rows
Illustrative daily row counts: 1,000 orders with an average of 3 lines, and 400 products sold that day.

Three kinds of fact table

Transaction

- **Grain:** one row per event, such as an order line
- **Measures:** additive quantities and amounts
- **Load:** insert new rows, rarely update
- **Risk:** very large tables

Periodic snapshot

- **Grain:** one row per entity per period
- **Measures:** balances and levels, semi-additive
- **Load:** insert a full period each run
- **Risk:** never sum a balance across time

Accumulating snapshot

- **Grain:** one row per process instance
- **Measures:** milestone dates and durations
- **Load:** update the row as milestones occur
- **Risk:** needs merge logic, plan for it

Dimensions and surrogate keys

```
dim_customer

create table dim_customer (
  customer_key bigint primary key, -- [1]
  customer_id varchar not null, -- [2]
  name varchar,
  segment varchar,
  city varchar,
  valid_from date, -- [3]
  valid_to date,
  is_current boolean
);
insert into dim_customer -- [4]
values (-1, 'n/a', 'Unknown', null, null,
       '1900-01-01', '9999-12-31', true);
```

1 Surrogate key

A meaningless integer owned by the warehouse. Stable when source ids change.

2 Natural key kept

The source id stays for lookups and traceability.

3 History columns

Support type 2 changes, shown in the next slides.

4 Unknown member

Key -1 catches facts whose dimension row has not arrived.

Slowly changing dimensions: the main types

Type	Behaviour on change	History	Use when
Type 0	Never change the value	Original only	Fixed attributes such as birth date
Type 1	Overwrite the value	None	Corrections and attributes nobody trends
Type 2	Add a new row with validity dates	Full	Attribute history affects analysis
Type 3	Keep the prior value in an extra column	One previous value	A simple before-and-after view
Type 4	Split fast-changing attributes into a mini-dimension	Kept in the mini-dimension	Attributes that change very often
Type 6	Combine types 1, 2 and 3	Full, plus current values	Report by then-values and now-values

Type 2 is the workhorse. Choose types per attribute, not per table.

Type 2 in practice: a customer moves city

customer_key	customer_id	city	valid_from	valid_to	is_current
101	C-77	Leeds	2023-01-10	2024-03-06	false
245	C-77	Manchester	2024-03-07	9999-12-31	true

1 Close the old row

Set valid_to to the day before the change and is_current to false.

2 Open a new row

Assign a new surrogate key, new values and a far-future valid_to.

3 Facts keep their key

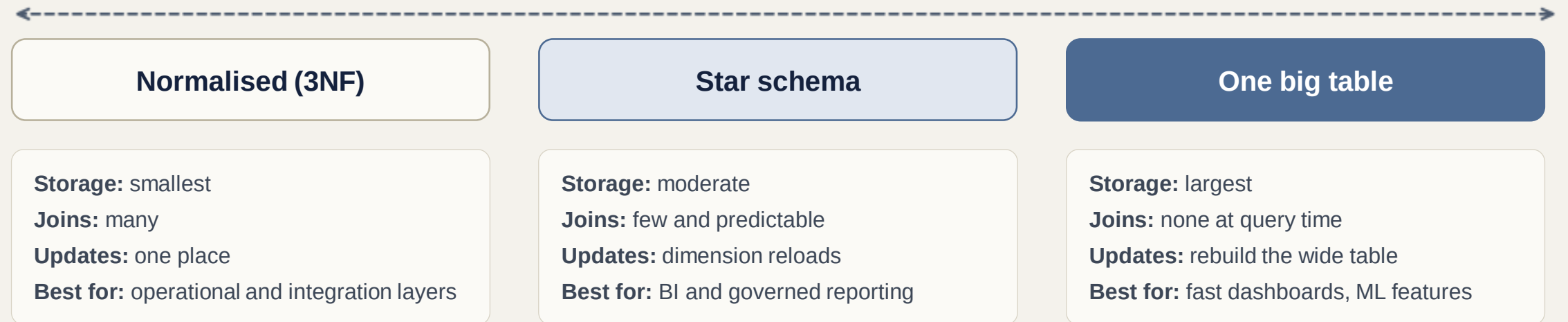
Orders placed while she lived in Leeds still point at key 101.

Detect changes by comparing a hash of the tracked columns, then close and insert in one atomic MERGE.

Normalised, dimensional and one big table

Less redundancy, more joins

More redundancy, fewer joins



Most platforms use all three: normalised staging, star-schema marts, and wide tables for specific consumers.

Three modelling traditions

Kimball: dimensional

- Star schemas per business process
- Conformed dimensions link marts
- Bottom-up: deliver value early
- Excellent for BI and self-service

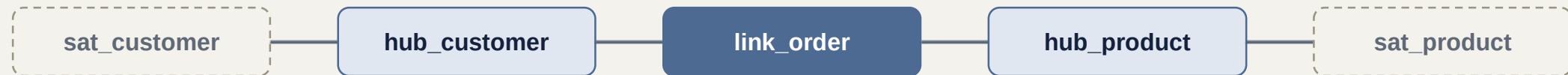
Inmon: enterprise

- Normalised enterprise warehouse first
- Marts derived from the core
- Top-down, strong integration
- Heavier upfront design

Data Vault

- Hubs, links and satellites
- Insert-only, auditable history
- Absorbs source change with little rework
- Needs a presentation layer for BI

Data Vault in miniature: hubs hold business keys, links hold relationships, satellites hold history



PART 06 · LEVEL 3 · INTERMEDIATE

Extraction deep dive: reading sources safely

Extraction fixes the freshness, completeness and source load of every pipeline. Learn the patterns, the traps in each, and how to choose.

Patterns

Full and incremental

Watermarks

CDC

Change events

APIs

Backoff

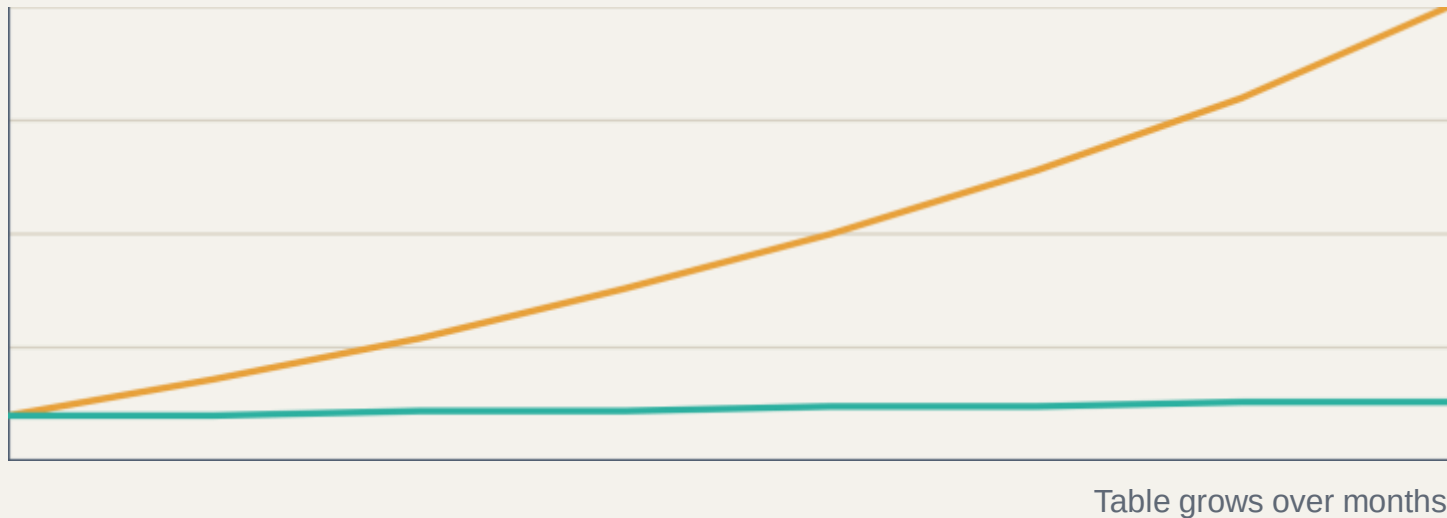
Files and events

Extraction patterns at a glance

Pattern	Mechanism	Freshness	Source impact	Sees deletes?
Full snapshot	Read the whole table each run	Batch	High	By comparison
Incremental timestamp	Filter on an updated_at watermark	Batch	Low to medium	No
Incremental key	Filter on an increasing id	Batch	Low	No, misses updates
Snapshot diff	Compare hashes with a prior copy	Batch	High	Yes
Log-based CDC	Read the database transaction log	Seconds	Very low	Yes
Trigger-based CDC	Triggers write to change tables	Seconds	Medium	Yes
Event streaming	Consume events the source publishes	Seconds	Low	If events exist

Pick the cheapest pattern that meets the freshness need and can still detect deletes if they matter.

Full extraction against incremental extraction



Full extraction: reads the whole table

Incremental: reads only the changes

Full extraction

Run time and source load grow with table size, not with the amount of change.

Incremental

Cost tracks the daily change volume, so it stays flat while the table grows.

Full is still fine for

Small reference tables and sources with no change tracking. Keep the design simple.

Illustrative shapes, not measurements.

Incremental extraction with a watermark

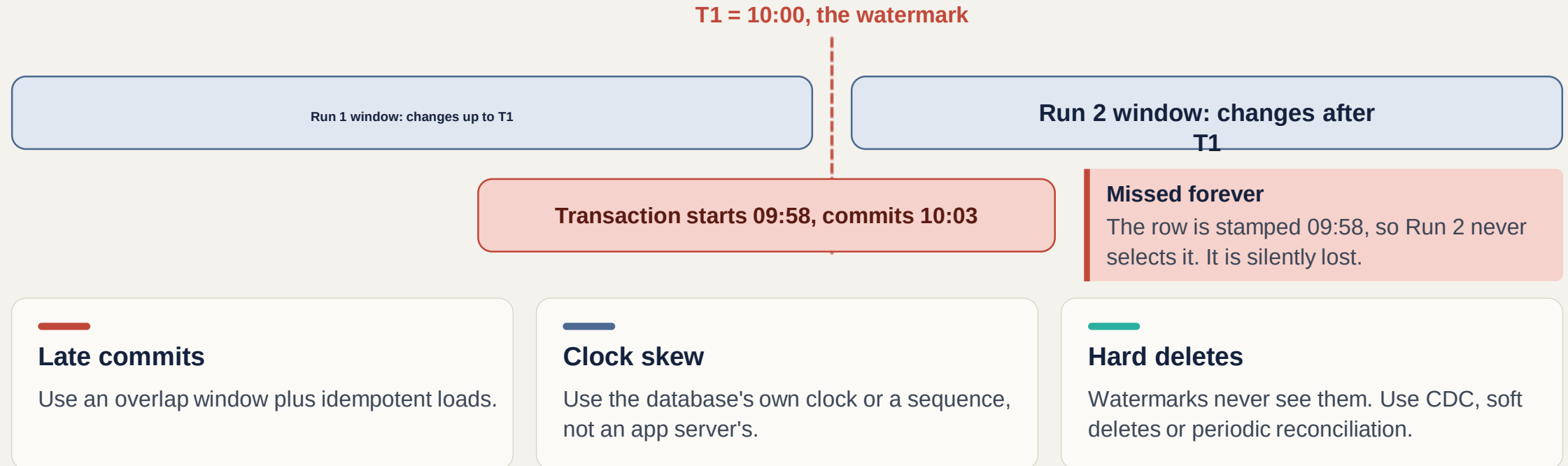
```
incremental_orders.sql

select *
from orders
where updated_at >= :last_watermark -- [1]
- interval '5 minutes'
  and updated_at < :run_started_at -- [2]
order by updated_at; -- [3]

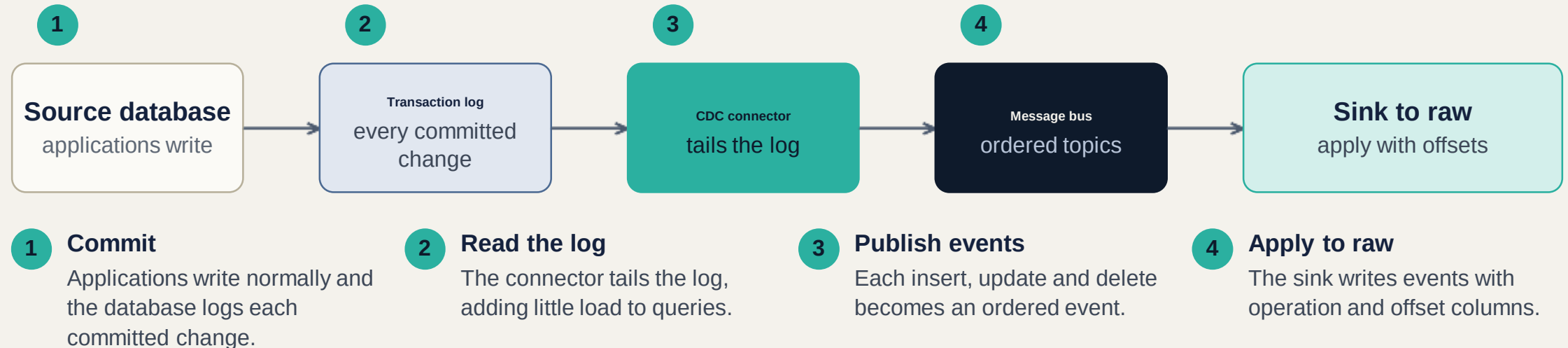
-- after a successful load only -- [4]
update etl_state
set high_watermark = :run_started_at
where pipeline = 'orders';
```

- 1 Overlap window**
Re-read a few minutes to catch slow commits. Loads must be idempotent.
- 2 Upper bound**
A fixed end time keeps the window stable if rows change mid-run.
- 3 Deterministic order**
Makes resumable reads and debugging predictable.
- 4 Advance last**
Move the watermark only after the load has committed.

Watermark pitfalls



Change data capture: reading the log



Log-based CDC captures deletes and intermediate states, which query-based methods cannot.

Three ways to capture change

Style	How it works	Deletes	Source impact	Watch out for
Log-based	Reads the transaction log	Yes	Very low	Log access, retention and permissions
Trigger-based	Triggers copy changes to a table	Yes	Medium, extra write cost	Trigger upkeep and schema changes
Query-based	Polls on timestamp or version	Only if soft	Low to medium	Misses changes between polls
Snapshot diff	Compares full snapshots	Yes	High	Heavy on large tables

Managed log-based tools include Debezium, AWS DMS and Google Datastream, among others. Check current source-database support.

Anatomy of a change event

change event (simplified)

```
{  
  "op": "u", // [1]  
  "source": { "table": "orders",  
              "lsn": 88123411 }, // [2]  
  "ts_ms": 1710000123456, // [3]  
  "before": { "id": 1001, "status": "NEW" },  
  "after": { "id": 1001, "status": "SHIPPED" } // [4]  
}
```

1 Operation

c create, u update, d delete, r snapshot read.

2 Position

Table plus log sequence number: the offset for resuming.

3 Timestamp

When the change committed. Use it to order and deduplicate.

4 Before and after

Full images let sinks apply updates and audit what changed.

Extracting from APIs

```
extract_orders.py

client = RetryingClient(max_tries=5) # [4]

def fetch_all(client, since):
    cursor = None
    while True:
        page = client.get('/orders',
after=cursor, # [1]
updated_since=since) # [2]
        yield from page['data']
        cursor = page.get('next_cursor')
        if not cursor: # [3]
            break
```

1 Cursor paging

Cursors survive inserts. Page numbers can skip or repeat rows.

2 Filter at the source

Ask only for changes since the last watermark.

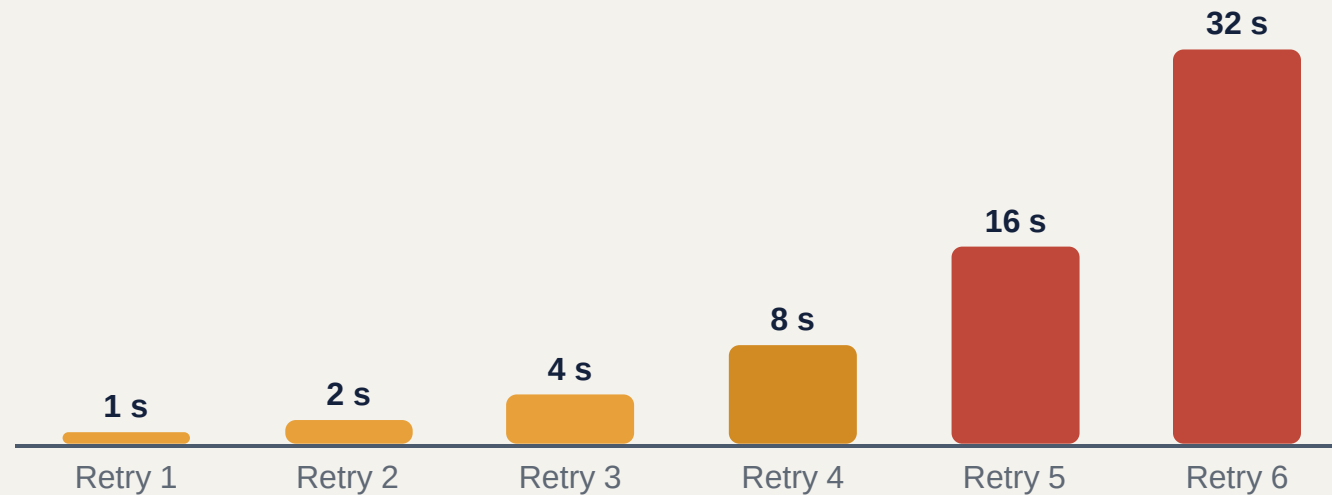
3 Explicit end

Stop on a missing cursor, never on a short page.

4 Retry wrapper

Handle timeouts, 429 and 5xx responses with backoff.

Rate limits, retries and backoff



Exponential

Doubling the wait gives a struggling API time to recover.

Jitter

Add random spread so many clients do not retry in lockstep.

Respect Retry-After

If the API states a wait time, use it before any formula.

Cap attempts and maximum wait, then fail loudly so the orchestrator can alert. Illustrative waits before jitter.

Files, events and legacy sources

Files

- Read a manifest, not a directory listing
- Verify size or checksum before parsing
- Move processed files, never delete first
- Expect late, duplicate and partial drops

Event streams

- Track consumer offsets per partition
- Commit offsets after the load commits
- Deduplicate on event id
- Plan for replay and reordering

Legacy exports

- Document the fixed-width layout explicitly
- Handle encodings such as EBCDIC or Latin-1
- Land the file byte for byte before parsing
- Reconcile control totals with the sender

Whatever the source, land it unchanged first and record exactly what was read.

PART 07 · LEVEL 4 · ADVANCED

Transformation deep dive: SQL that scales

Move from concepts to craft: cleansing, joins, windows, incremental models, idempotency, testing and performance.

Taxonomy

Deduplication

Standardising

Join fan-out

Windows

Incremental models

Idempotency

Testing

Late data

Performance

A taxonomy of transformations



Cleansing

Nulls, duplicates, outliers, bad records.
COALESCE, NULLIF, ROW_NUMBER.



Standardisation

One format, unit and vocabulary. CAST, TRIM, CASE, mapping tables.



Integration

Join and reconcile sources through conformed keys. JOIN, UNION, crosswalks.



Enrichment

Add context from reference data. Lookups, geocoding, classification.



Restructuring

Change the shape: pivot, unpivot, flatten nested data. UNNEST, JSON functions.



Aggregation and derivation

Roll up and compute metrics. GROUP BY, window functions, ratios.

Cleansing: deduplicate with a window

```
dedupe_orders.sql
with ranked as (
  select *,
    row_number() over (
      partition by order_id -- [1]
      order by updated_at desc, -- [2]
              _loaded_at desc
    ) as rn
  from stg.orders
)
select * from ranked
where rn = 1; -- [3]
-- [4] some engines offer QUALIFY
```

- 1 Define the entity**
Partition by the business key that must be unique.
- 2 Define the winner**
Order by change time, then load time, so ties are deterministic.
- 3 Keep one row**
Every other row is a superseded version.
- 4 Shortcut**
QUALIFY filters window results directly where supported.

Standardisation: one format for every value

Concern	Raw input	Standard output	Rule
Dates	2024/3/7 and 07-Mar-24	2024-03-07	ISO 8601, explicit source format
Time zones	09:00 with no zone	2024-03-07T09:00Z	Convert to UTC, keep local zone
Currency	1,250.00 and £1250	1250.00 and GBP	Parse, split amount and code
Country	UK, U.K., United Kingdom	GB	Map to ISO 3166 codes
Phone	(0161) 555 0100	+441615550100	E.164 international format
Text	'ACME Corp. '	acme corp	Trim, case-fold, normalise Unicode

Keep the mapping tables in version control. A silent change to a lookup is a silent change to every report.

Joins: the classic source of wrong numbers

orders

order_id	amount
1001	100

order_items

order_id	item
1001	A
1001	B

join on order_id

joined result

order_id	item	amount
1001	A	100
1001	B	100

SUM(amount) = 200
the true total is 100

Fan-out

A one-to-many join repeats parent values on every child row.

Fix

Aggregate items first, or sum at the parent's grain.

Before trusting a join, check that row counts and key uniqueness match what you expect on both sides.

Window functions for analytical transforms

```
customer_running_totals.sql

select
  customer_id, order_date, amount,
  sum(amount) over ( -- [1]
    partition by customer_id -- [3]
    order by order_date
    rows unbounded preceding -- [4]
  ) as running_total,
  lag(amount) over ( -- [2]
    partition by customer_id
    order by order_date
  ) as prev_amount
from fct_orders;
```

- 1 Running total**
Aggregate over an ordered frame without collapsing rows.
- 2 Look back a row**
LAG and LEAD compare a row with its neighbours.
- 3 Partition**
The window restarts for each value, such as each customer.
- 4 State the frame**
Default frames differ between engines and functions. Be explicit.

Incremental models

```
fct_orders.sql (dbt-style)
{{ config(materialized='incremental',
          unique_key='order_id') }} -- [1]

select order_id, customer_id,
       status, updated_at
from {{ ref('stg_orders') }} -- [2]
{% if is_incremental() %}
where updated_at > -- [3]
    (select max(updated_at) from {{ this }})
{% endif %}
-- [4] dbt run --full-refresh
```

dbt-style syntax shown. Other tools express the same idea differently.

- 1 Unique key**
Existing rows with the same key are updated, new ones inserted.
- 2 ref() builds the graph**
Dependencies are declared in code, so build order is automatic.
- 3 Incremental filter**
Later runs process only rows newer than the target's latest.
- 4 Full refresh**
Keep a way to rebuild from scratch when the logic changes.

Idempotency: safe to run twice

Unsafe: append on every run



Safe: merge or overwrite by key or partition



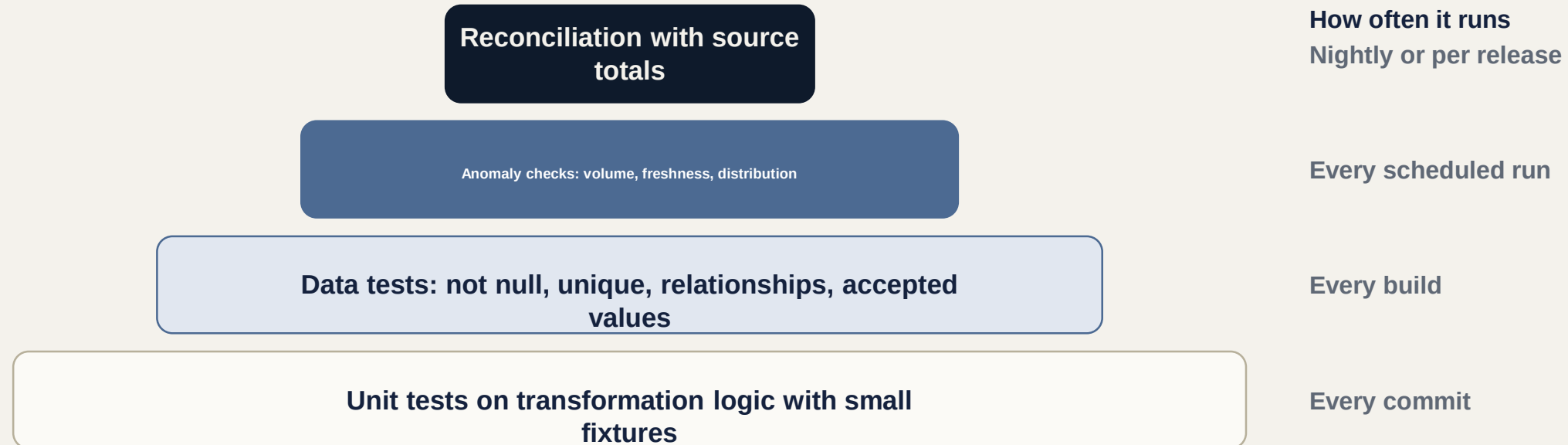
Design every load so that running it twice leaves the same result as running it once. Retries then become harmless.

Materialisation choices

Option	Built as	Freshness	Build cost	Use when
View	Saved query, run on read	Always current	None	Light logic, small data
Table	Rebuilt fully each run	As of last run	Full rebuild	Heavy logic, moderate data
Incremental	New or changed rows merged	As of last run	Proportional to change	Large, append-heavy data
Snapshot	Row versions with validity dates	As of last run	Proportional to change	Track history, type 2
Materialized view	Engine-maintained result	Near current	Engine-managed	Repeated heavy aggregations

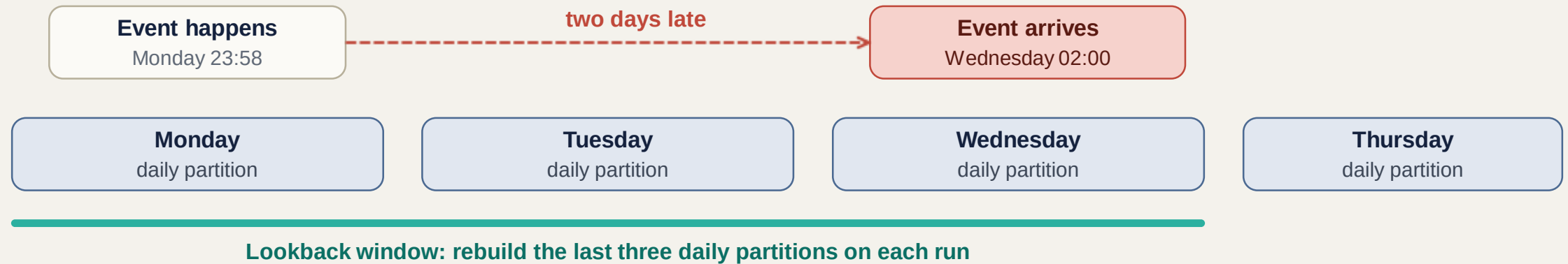
Start with views, promote to tables when queries slow down, and to incremental only when full rebuilds become too costly.

Testing transformations: a pyramid



Push most checks to the cheap, fast bottom layers. Keep the expensive top layer small and high-value.

Late and out-of-order data

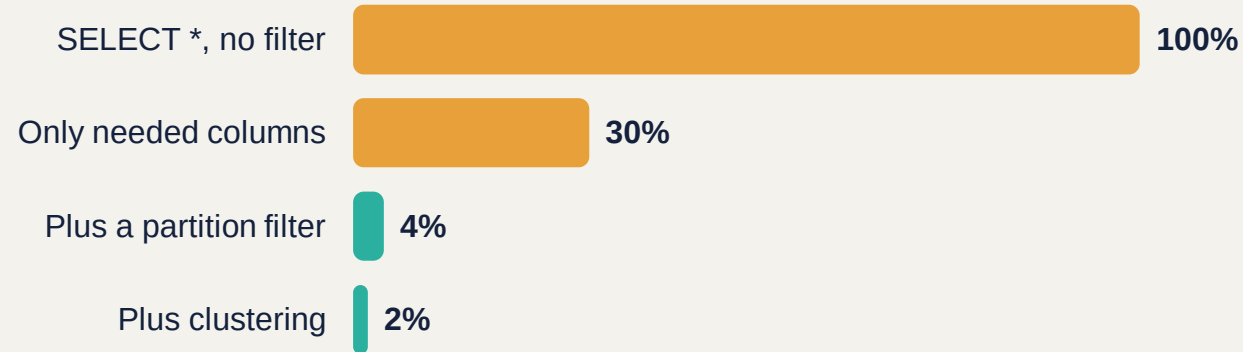


Sizing the window

Choose the window from measured lateness, for example the 99th percentile of arrival delay, and monitor it. Anything later goes to a slower correction path.

Separate event time (when it happened) from processing time (when you saw it). Partition by the first, load by the second.

Transformation performance: read less data

**Select needed columns**

Columnar storage reads only the columns you name.

Filter on partitions

Partition pruning skips whole date ranges.

Cluster or sort keys

Co-locate related rows so filters read fewer blocks.

Illustrative share of data scanned for one query as each technique is added. Real savings depend on the engine and the data layout.

On platforms that charge for compute or bytes scanned, reading less data cuts both time and cost.

PART 08 · LEVEL 4 · ADVANCED

Loading deep dive: publishing data safely

Loading is where correctness becomes visible to users. Learn the strategies, atomic publishing, partitioning, backfills and failure handling.

Strategies

Bulk loading

Merge

Atomic swap

Partitioning

Backfills

Deletes

Effectively-once

Quarantine

Load strategies at a glance

Strategy	What it does	Updates	Deletes	Best for
Truncate and load	Replace the whole table	Yes	Yes	Small and reference tables
Append only	Insert new rows	No	No	Immutable events and logs
Upsert (merge)	Update matches, insert the rest	Yes	Optional	Mutable entities with keys
Delete and insert	Remove a slice, insert fresh rows	Yes	For the slice	Batch or partition reloads
Partition overwrite	Replace whole partitions atomically	Yes	Per partition	Large time-partitioned facts
SCD type 2	Close old rows, insert new versions	As history	Optional	Tracked dimensions

Choose per table. A single pipeline usually mixes several strategies.

Bulk loading: files beat row inserts



Why bulk wins

Set-based file loads use parallel readers and skip per-row overhead. Prefer columnar files such as Parquet.

COPY INTO syntax shown. Other engines use COPY or LOAD DATA.

```
copy into raw.orders
from @landing/orders/2024-03-07/ -- one batch folder
file_format = (type = parquet) -- columnar, typed files
on_error = abort_statement; -- fail fast, never half-load
```

Relative throughput is illustrative. Actual gains depend on the platform, file size and parallelism.

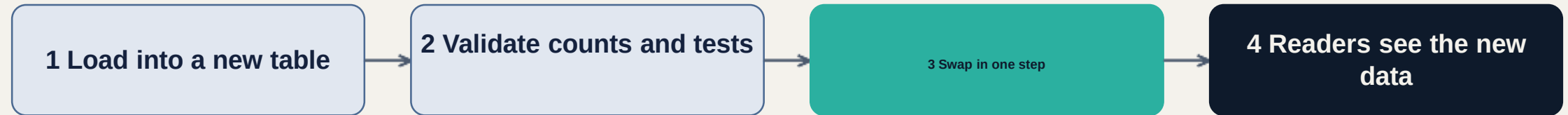
Upsert with MERGE

```
merge_customer.sql

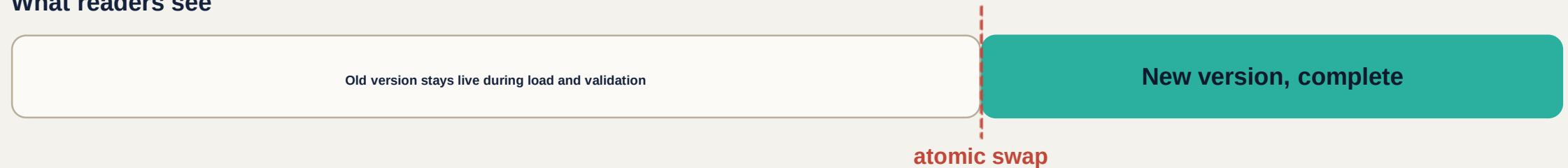
merge into dim_customer as t
using stg_customer as s
  on t.customer_id = s.customer_id -- [1]
when matched and t.row_hash <> s.row_hash then
  update set name = s.name, -- [2]
           city = s.city,
           row_hash = s.row_hash
when not matched then
  insert (customer_id, name, city, row_hash)
  values (s.customer_id, s.name,
         s.city, s.row_hash); -- [3]
-- [4] source must be unique on the key
```

- 1 Match on the business key**
Each side needs exactly one row per key, or the match is ambiguous.
- 2 Update real changes only**
A row hash avoids rewriting unchanged rows.
- 3 Insert the rest**
New keys arrive as new rows.
- 4 Deduplicate the source**
Many engines error when two source rows match one target row.

Atomic publishing with a staging swap



What readers see



Readers never see a half-loaded table. Engines offer this as rename, swap, create-or-replace, or by repointing a view.

Partitioning and clustering



Whole partitions the engine can skip without reading a byte

Partition on the filter

Use the column queries filter by, usually a date.

Avoid over-partitioning

Aim for large partitions. Thousands of tiny ones slow planning and loads.

Cluster inside

Sort or cluster on a second key such as customer_id.

Backfills and historical reloads

1

Scope the range

Choose the date range and tables affected. Estimate volume and cost first.

2

Pin the logic

Fix the code version so old and new data follow the same rules.

3

Rebuild into a shadow target

Write to shadow tables so production is untouched during the run.

4

Reconcile

Compare counts and key totals against the old data and the source.

5

Swap and record

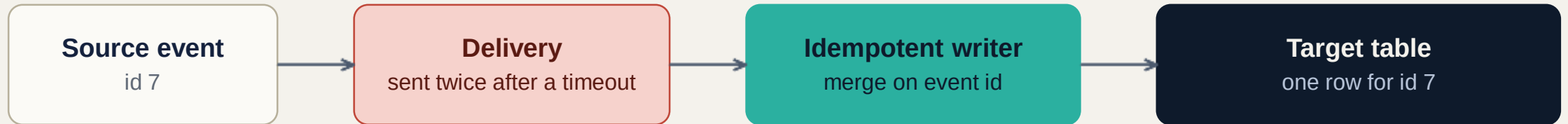
Promote atomically, then log what changed and why.

Handling deletes

Source signal	Approach	Trade-off
Soft-delete flag	Filter on or propagate is_deleted	Easy, but every consumer must respect the flag
CDC delete event	Apply the delete or write a tombstone	Prompt and accurate, needs log-based capture
Full snapshot compare	Anti-join against the last snapshot	Reliable, expensive on large tables
Periodic reconciliation	Compare key lists on a schedule	A cheap safety net, but results arrive late
No signal at all	Rebuild the table on a schedule	Simple, and cost grows with table size

Decide deliberately whether a deleted source row should vanish or stay behind with a deleted marker for history.

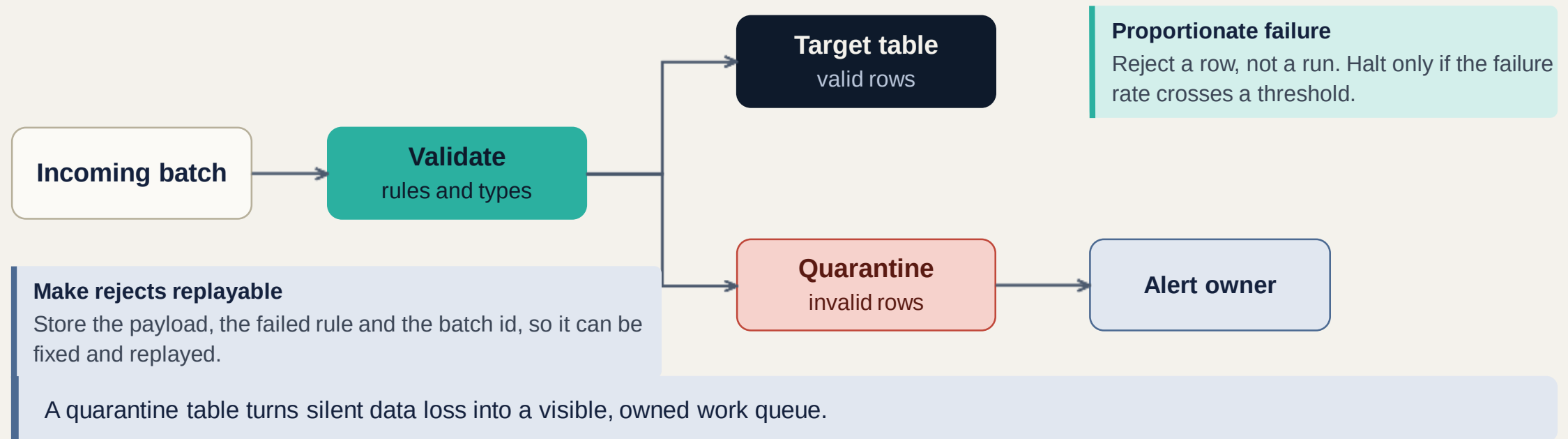
Atomic loads and effectively-once results



- 1 At-least-once delivery**
It is the realistic guarantee. Networks and clients retry.
- 2 Idempotent writes**
Merge on a stable key so duplicates change nothing.
- 3 Atomic commits**
A batch applies completely or not at all.

Exactly-once delivery is rare. Effectively-once results come from at-least-once delivery plus idempotent, atomic writes.

Quarantine: loading around bad rows



Orchestration: running pipelines reliably

Steps must run in order, on time, and recover when things break. Orchestration turns scripts into an operated system.

Responsibilities

DAGs

Pipelines as code

Triggers

Retries and SLAs

Backfills

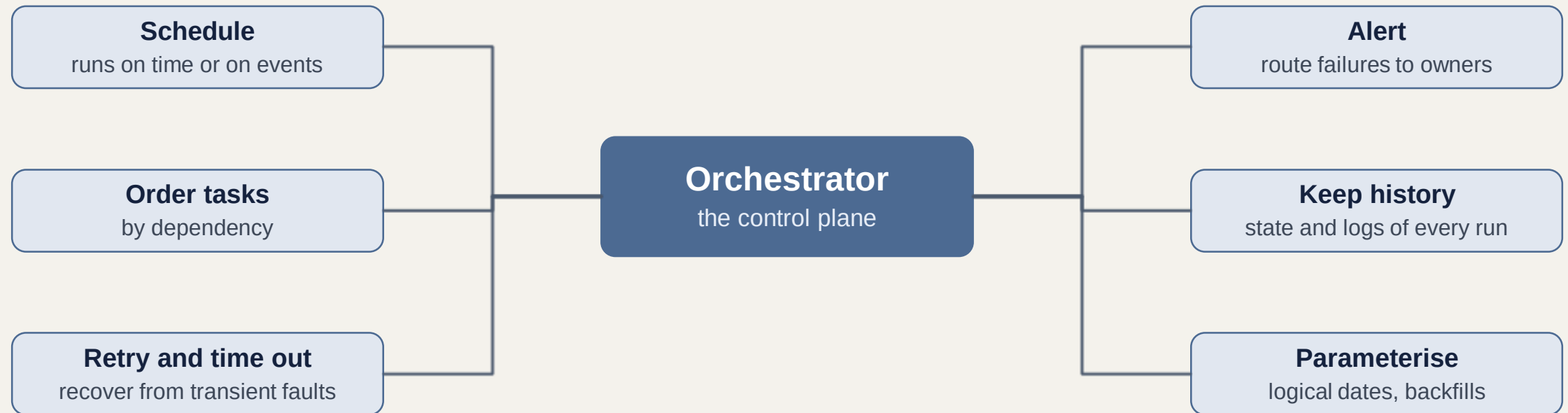
Logical dates

Dependencies

Tools

Failure patterns

What an orchestrator does

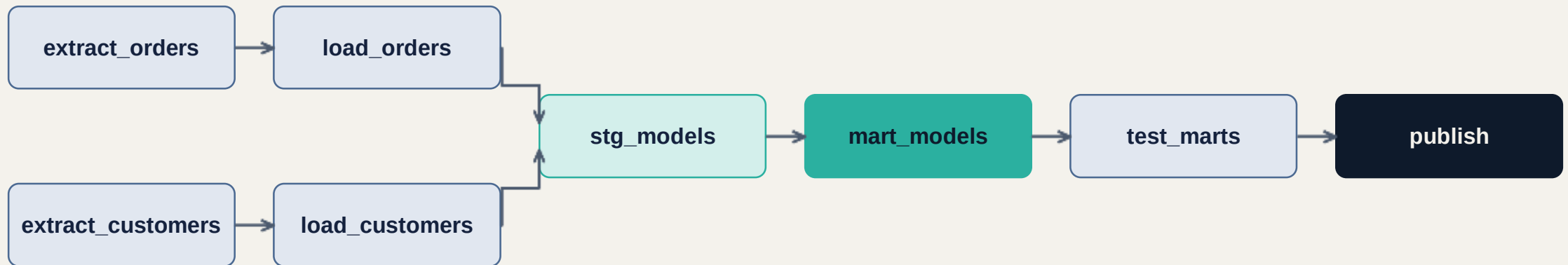


Orchestrators coordinate work. The heavy processing should run in the warehouse, Spark or another engine, not in the scheduler.

DAGs: tasks and dependencies

Parallel branches

Fan-in



1 Directed

Edges point one way. A task runs after everything it depends on.

2 Acyclic

No loops. A cycle would mean a task waits for itself.

3 Parallel where possible

Independent branches run at once to cut elapsed time.

A pipeline defined as code

```
dags/daily_orders.py

@dag(schedule="0 2 * * *", # [1]
      start_date=datetime(2026, 1, 1),
      catchup=False, # [2]
      default_args={"retries": 3}) # [3]
def daily_orders():
    raw = extract_orders()
    load = load_to_raw(raw)
    build = run_dbt('marts.orders')
    load >> build # [4]

daily_orders()
```

- 1 Schedule**
A cron expression: here 02:00 every day.
- 2 Catch-up**
When off, missed intervals do not run by themselves.
Decide on purpose.
- 3 Default retries**
Each task retries three times before it fails.
- 4 Dependency**
The right-shift operator says load runs before build.

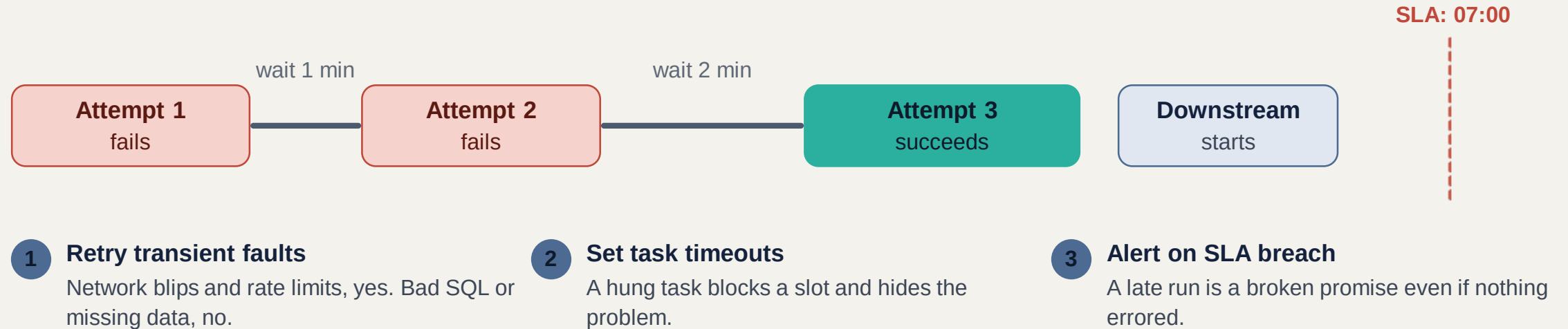
Airflow-style syntax shown. Dagster and Prefect express the same ideas differently.

Scheduling models: time, event and data-aware

Trigger	How it fires	Strength	Weakness
Time-based	Cron expression or interval	Simple and predictable	Runs even if upstream data is late
Event-based	File arrival, message or webhook	Reacts to real arrivals	Needs reliable events and deduplication
Data-aware	An upstream dataset is updated	Encodes the true dependency	Needs asset tracking across teams
Sensor or polling	Wait until a condition holds	Bridges systems without events	Occupies slots and can stall
Manual	A person or an API call	Full control for backfills	Unsuitable for routine runs

Prefer triggers that express real dependencies. A clock is only a proxy for 'the data is ready'.

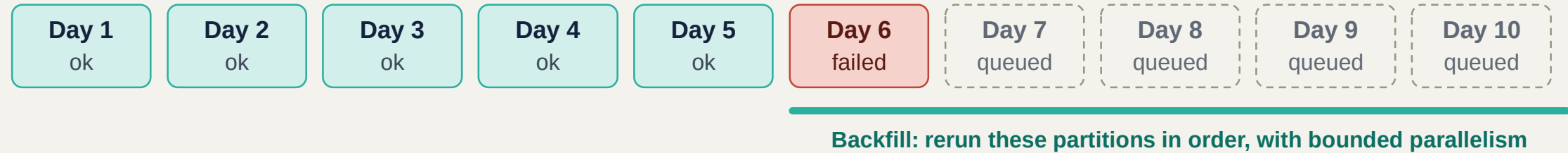
Retries, timeouts and SLAs



Retries add resilience only when tasks are idempotent. Otherwise a retry creates the next incident.

Backfill and catch-up semantics

One partition per logical date



Idempotent tasks

A rerun replaces a partition. It never adds to it.

Bounded parallelism

Cap concurrent backfill runs so sources and warehouses stay healthy.

Dependency order

Rebuild upstream partitions before the downstream ones.

Logical dates: why now() is a bug

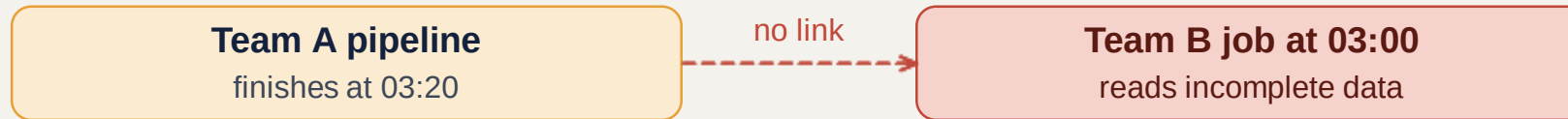
```
daily_sales.sql
begin;
delete from marts.daily_sales
where sales_date = :ds; -- [1]

insert into marts.daily_sales
select :ds as sales_date, -- [2]
        product_id,
        sum(amount) as revenue
from stg.orders
where order_date = :ds -- [3]
group by product_id;
commit; -- [4] rerun any past :ds
```

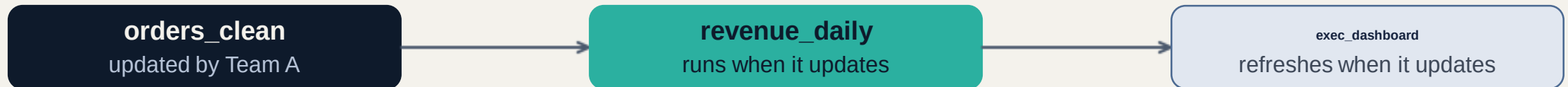
- 1 Replace the slice**
Delete then insert the same date so reruns converge.
- 2 Never use now()**
Wall-clock time changes on every rerun. The logical date does not.
- 3 One parameter**
Filter, label and target partition all use the same date.
- 4 Backfills for free**
Run the same task for any past date by passing it in.

Dependencies across teams

Fixed clocks: fragile



Data-aware: the dependency is the dataset



Model datasets, not clocks, as the unit of dependency. Pair it with data contracts so upstream changes are visible.

The orchestrator landscape

Tool	Model	Strengths	Consider
Apache Airflow	Python DAGs, scheduler, operators	Huge ecosystem, mature, widely known	Heavier to run, task-centric
Dagster	Software-defined assets	Asset lineage, testing, local development	Newer conventions to learn
Prefect	Python flows and tasks	Light setup, dynamic workflows	Fewer prebuilt integrations
Cloud-native	ADF, AWS Step Functions, Databricks Workflows	Managed, integrated with the cloud	Portability and lock-in
dbt Cloud jobs	Scheduled dbt runs	Simple for SQL-only projects	Limited beyond dbt

Summaries are simplified. Verify current capabilities and pricing before choosing.

Failure-handling patterns



Retry with backoff

Repeat transient failures with growing, jittered waits.



Circuit breaker

Stop calling a failing dependency for a cooling-off period.



Branch or skip

Route around optional tasks so one failure does not block all.



Dead-letter queue

Park items that keep failing for later inspection.



Approval gate

Pause before risky steps such as backfills or destructive changes.



Alert routing

Send each failure to the owning team with a runbook link.

Quality and governance: earning trust

A fast pipeline that produces wrong numbers is worse than a slow one. Build tests, contracts, observability, lineage and access control into the design.

Quality dimensions

Quality gates

Tests as code

Contracts

Observability

Lineage

Access control

Privacy

Catalog

Incidents

The dimensions of data quality



Accuracy

Values match reality. Check that totals reconcile with the source.



Completeness

Required data is present. Check null rates and row counts.



Consistency

The same fact has the same value everywhere. Compare cross-system totals.



Timeliness

Data is fresh enough for its use. Check freshness against the SLA.



Validity

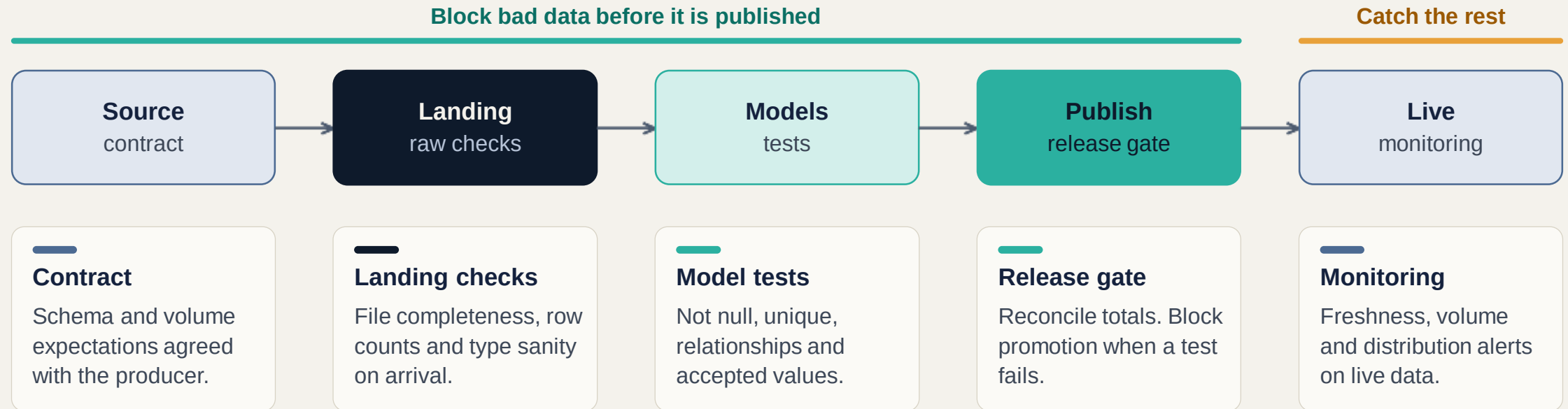
Values follow the rules and formats. Check accepted values and ranges.



Uniqueness

One row per real-world entity. Check unique keys and duplicate rates.

Quality gates along the pipeline



Fail early and cheaply. A defect caught at landing costs a rerun. The same defect found in a board report costs trust.

Expressing tests as code

```
schema.yml (dbt-style)
models:
  - name: fct_orders
    columns:
      - name: order_id
        tests: [unique, not_null] # [1]
      - name: customer_id
        tests:
          - relationships: # [2]
              to: ref('dim_customer')
              field: customer_id
freshness: # [3]
  error_after: {count: 12, period: hour} # [4]
```

- 1 Uniqueness and completeness**
The key must be present and must never repeat.
- 2 Referential integrity**
Every fact points at an existing dimension row.
- 3 Freshness check**
Alert when data is older than the agreed limit.
- 4 Severity**
Separate errors that block from warnings that inform.

Data contracts

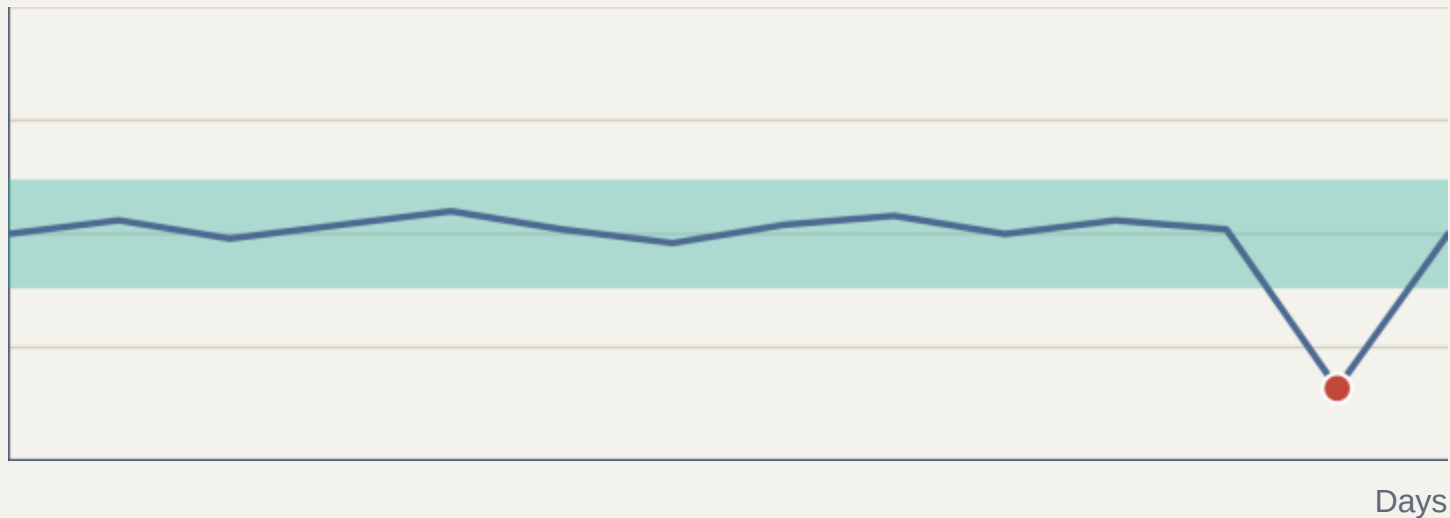
```
orders.contract.yaml

dataset: shop.orders
owner: team-checkout # [1]
schema:
  - name: order_id
    type: bigint
    required: true # [2]
  - name: status
    type: string
    allowed: [NEW, PAID, SHIPPED] # [3]
sla:
  freshness_minutes: 60 # [4]
```

Illustrative format. Open specifications and vendor formats exist for data contracts.

- 1 Named owner**
Someone to call when it breaks or must change.
- 2 Explicit schema**
Types and required fields are promises, not accidents.
- 3 Allowed values**
Adding a value is a versioned change consumers can plan for.
- 4 Service level**
Freshness and completeness targets that monitoring can enforce.

Observability: catching what tests miss



Daily row count with a learned expected range (illustrative)

Tests check rules you thought of. Monitors catch surprises you did not.

Expected range

Learned from history, adjusted for weekly seasonality.

Anomaly

Row count fell by two thirds. Alert the owner before dashboards break.

Five pillars

Freshness, volume, schema, distribution and lineage.

Lineage: knowing what depends on what



1 Impact analysis

Before changing a column, see every model and dashboard that uses it.

2 Root cause

When a number is wrong, walk upstream to the first faulty step.

3 Trust

Consumers can see exactly where a metric comes from.

Security and access control

Control	What it protects	Example
Role-based access	Who can read or write which schemas	Analysts read marts, engineers write raw
Column masking	Sensitive fields in query results	Emails are masked unless the role is privileged
Row-level security	Which rows a user may see	Regional managers see only their region
Encryption	Data at rest and in transit	Managed keys, TLS on every connection
Audit logging	Evidence of access and change	Query history retained and reviewed
Secrets management	Credentials for pipelines	Rotated tokens from a vault, none in code

Grant least privilege by role, and treat pipeline service accounts like people with access.

Privacy and compliance by design



Data minimisation

Extract and keep only what a defined purpose needs.



Classification

Tag columns by sensitivity, then drive masking and retention from the tags.



Retention and erasure

Set retention per dataset and support deletion across raw, marts and backups.



Purpose and records

Know why each dataset is processed and who may access it.

Engineering guidance, not legal advice. Involve your privacy and legal teams for regulatory requirements.

Metadata, catalogs and ownership

What a catalog records

- Business definitions and owners
- Schemas and column descriptions
- Lineage and usage
- Quality status and freshness
- Sensitivity tags and access policy

An ownership model

- Every dataset has an owner and a steward
- Owners approve breaking changes
- Consumers register their dependencies
- Deprecations follow a notice period
- Unowned, stale data is retired

A catalog nobody updates is worse than none. Generate metadata from pipeline code wherever possible.

Incident response for data pipelines

1

Detect

An alert or a user report opens an incident with a severity.

2

Contain

Pause downstream jobs and mark the affected tables as suspect.

3

Diagnose

Use lineage and run history to find the first faulty step.

4

Fix and backfill

Correct the cause, rebuild affected partitions and reconcile.

5

Learn

Hold a blameless review and add the test or monitor that would have caught it.

Architecture patterns: putting it together

Proven reference architectures for batch, streaming, lakehouse and domain-oriented platforms, plus the delivery and cost practices that keep them healthy.

Medallion

Lambda

Kappa

Lakehouse

Data mesh

Reverse ETL

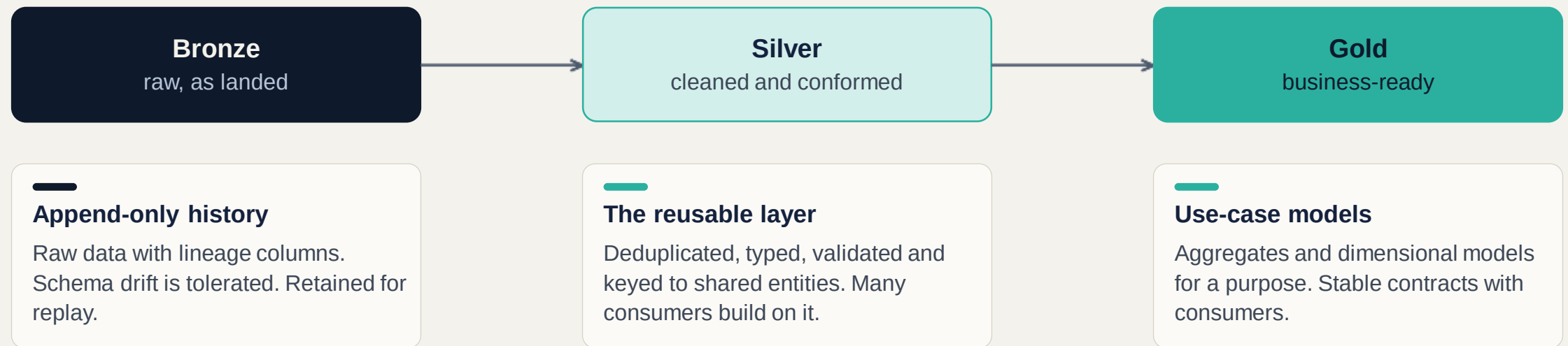
Stream windows

CI/CD

Cost

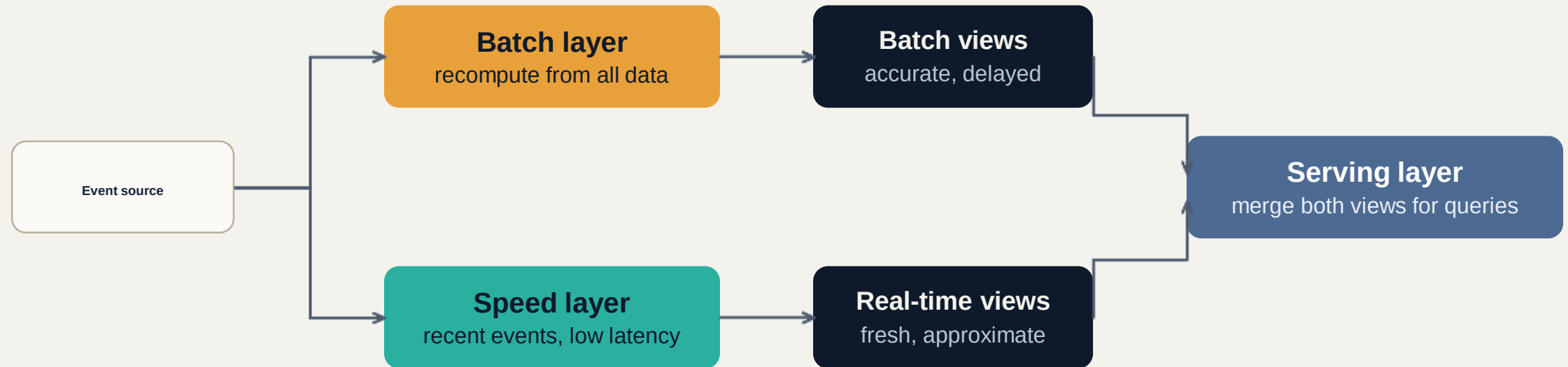
Medallion architecture: bronze, silver, gold

All transformation runs inside the platform



Medallion is ELT layering under different names: raw, staging and marts. The value is the shared vocabulary.

Lambda architecture: batch plus speed



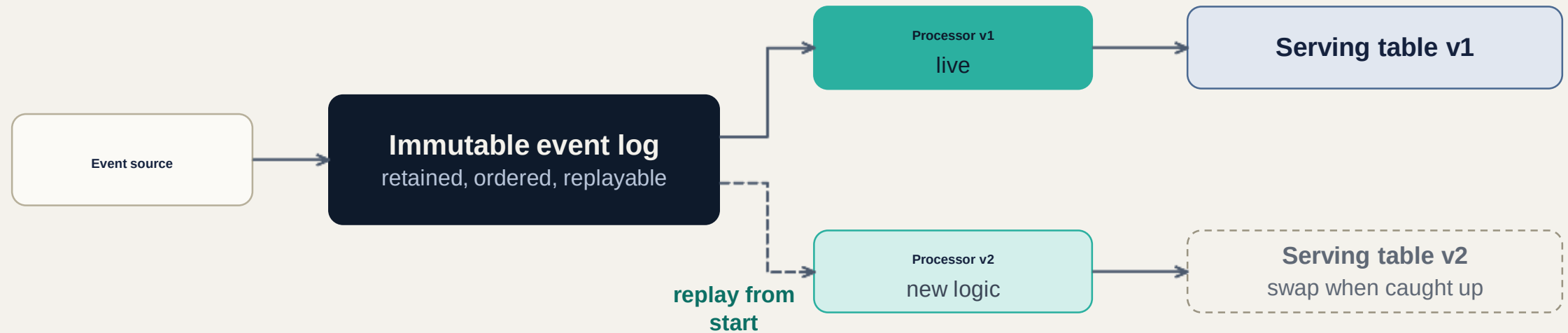
1 Strength

Accurate batch results combined with fresh recent data.

2 Cost

Two code paths that must produce the same answer.

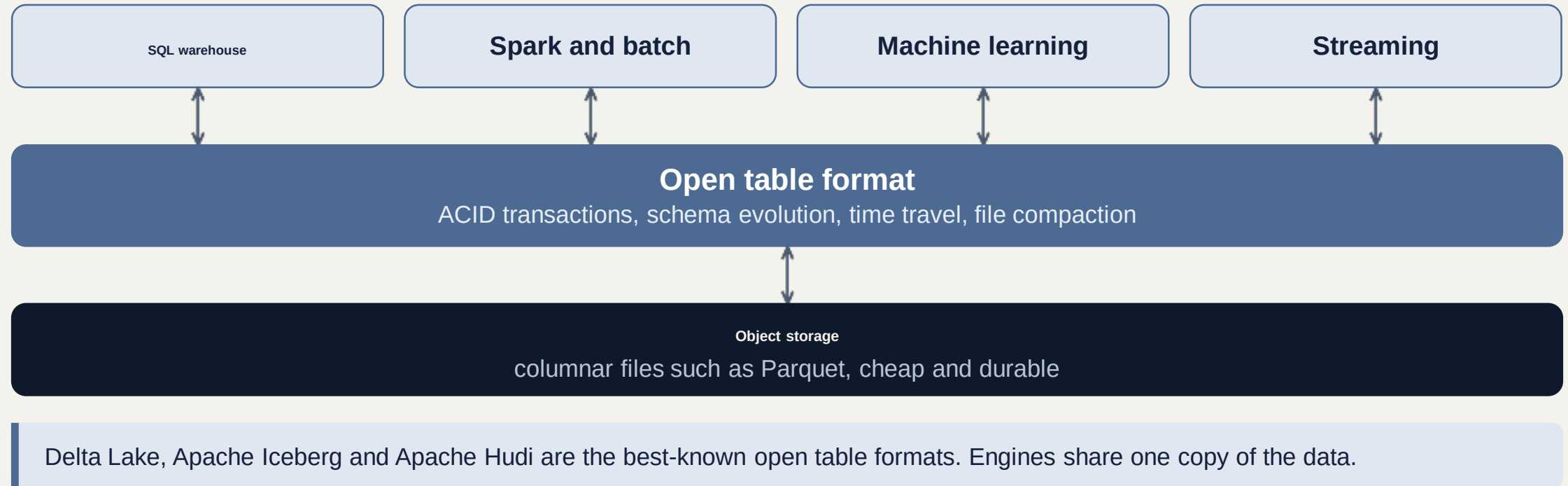
Kappa architecture: one stream, replayed



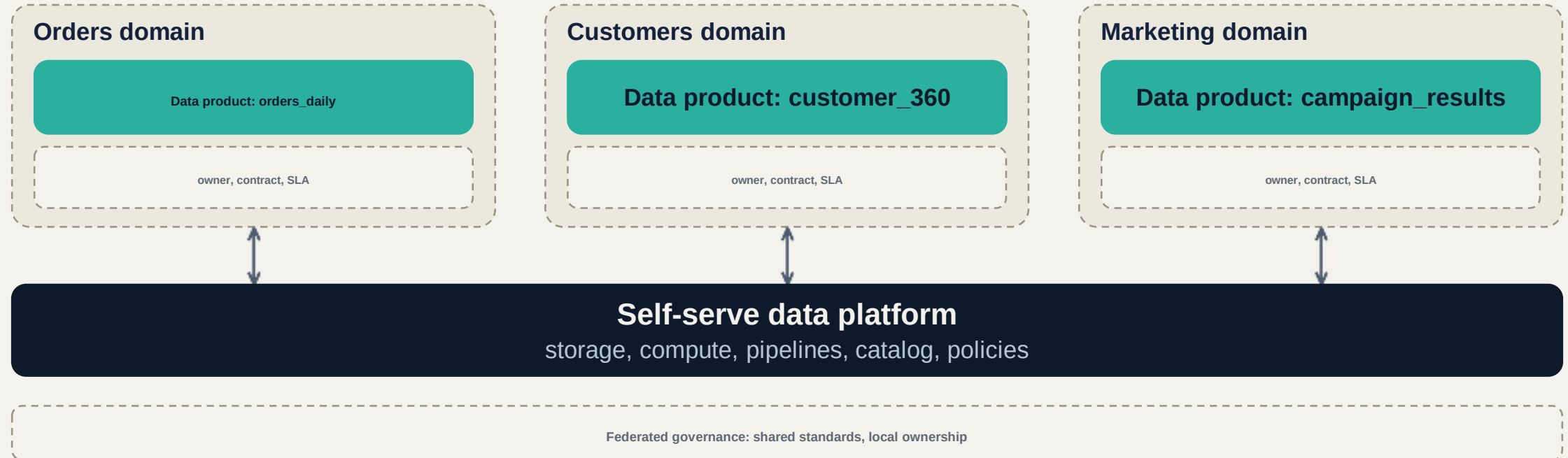
- 1 One code path**
Batch is just a replay of the stream, so logic exists once.

- 2 Needs long retention**
The log must be ordered, replayable and kept long enough.

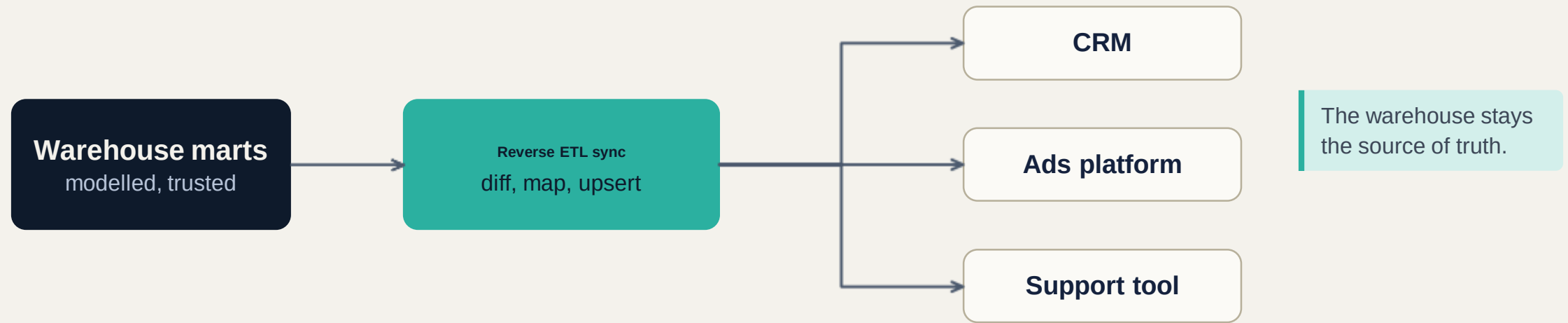
Lakehouse: warehouse features on open files



Data mesh: domains own data products



Reverse ETL: operational analytics



1 Sync changes only
Diff against the last sync so APIs receive minimal writes.

2 Respect limits
Batch, throttle and retry against destination rate limits.

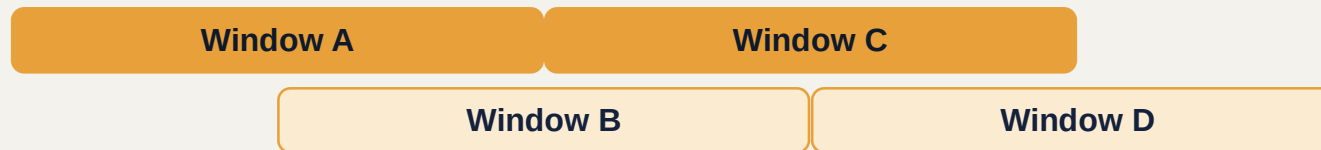
3 Upsert by key
Match on an external id so retries never create duplicates.

Streaming ETL: windows and state

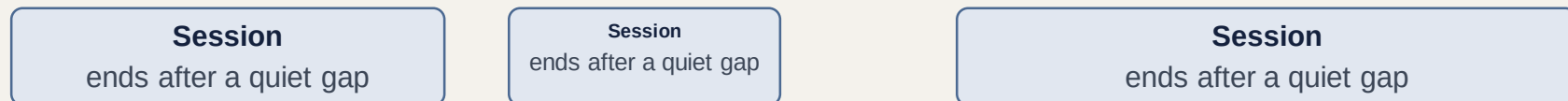
Tumbling



Sliding

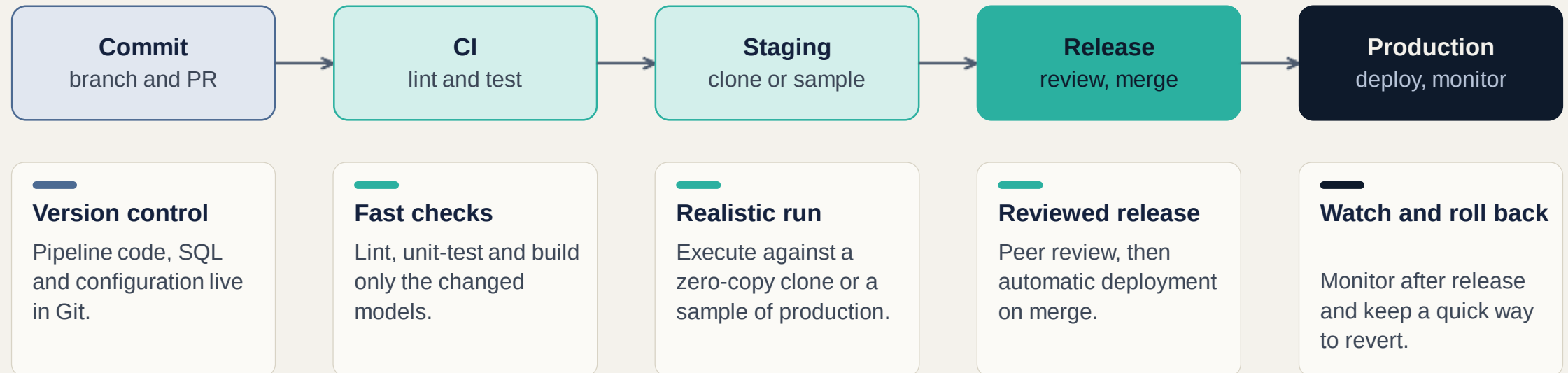


Session



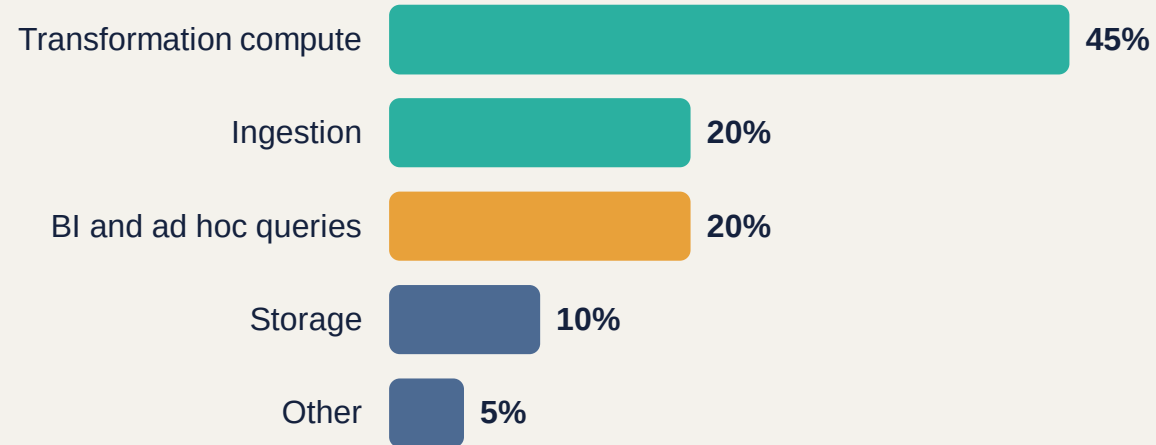
State grows with window size and key cardinality. Watermarks say how long to wait for late events before closing a window.

CI/CD for data pipelines



Treat pipelines as software: reviewed, tested, versioned and released.

Cost engineering: where the money goes



Compute levers

Right-size and auto-suspend compute. Use incremental models.

Query levers

Prune with partitions, select needed columns, cache repeated queries.

Governance levers

Tag spend by team, set budgets and alert on anomalies.

An illustrative split of monthly platform cost. Your own breakdown will differ, so measure it before optimising.

Optimise the biggest bar first, and add guardrails so a single bad query cannot dominate the bill.

Design practice: from requirements to architecture

Apply everything so far. A repeatable design method, three worked cases, sizing arithmetic, anti-patterns to avoid and a migration path.

Design method

E-commerce case

Finance case

IoT case

Requirements

Sizing

Anti-patterns

Migration

Review

A design method in seven questions

1 Purpose

Which decisions or products depend on this data?

2 Sources

Which systems, formats and change patterns are involved?

3 Freshness

How late can data be before it loses its value?

4 Volume

How big is it today, and how fast does it grow?

5 Sensitivity

What is personal or regulated, and where may it land?

6 Ownership

Who owns each dataset and who fixes it at night?

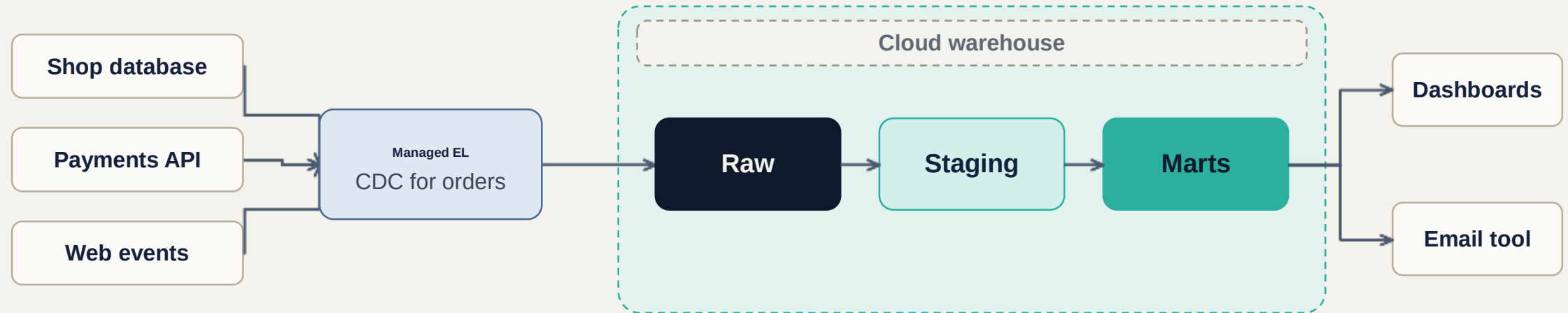
7 Failure

What breaks, how will you know, and how do you recover?

Then decide

Choose ETL, ELT or a hybrid only after the seven answers are written down.

Case 1: e-commerce analytics with ELT



1 Why ELT

A cloud warehouse, SQL-skilled analysts and rules that change weekly.

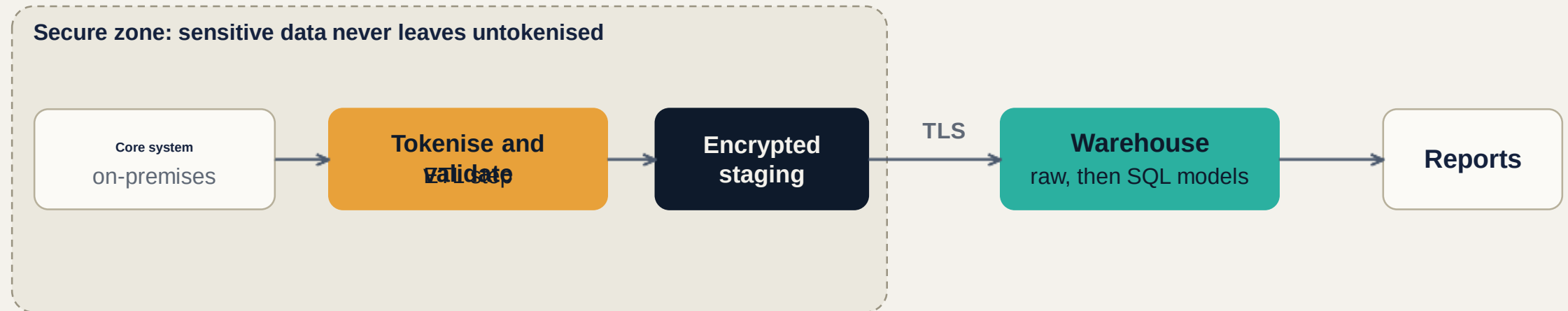
2 Freshness

Hourly loads satisfy marketing. CDC is used only for orders.

3 Watch

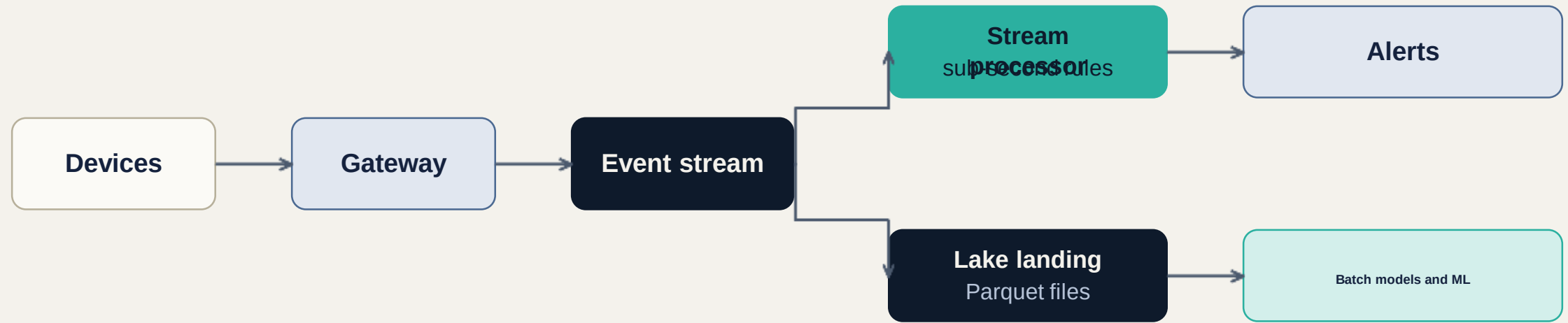
Personal data sits in raw. Mask by policy and honour deletion requests.

Case 2: regulated finance with ETLT



- 1 Tokenise first**
Sensitive fields are replaced before any data leaves the zone.
- 2 Audit by design**
Lineage, access logs and retained raw answer regulatory questions.
- 3 Model after load**
Business logic runs as SQL in the warehouse, keeping the ETL step small.

Case 3: IoT telemetry with hot and cold paths



1 Hot path

Sub-second rules, such as overheating alerts, run on the stream.

2 Cold path

Everything also lands in the lake for batch analytics and training.

3 Late data

Devices reconnect after outages. Partition by event time and use a lookback.

From requirements to design decisions

Requirement	Typical target	Design implication
Freshness	Hourly	Hourly incremental ELT or micro-batch. No streaming needed
Freshness	Under 10 seconds	Streaming or CDC into a low-latency store
Volume	Gigabytes per day	One warehouse copes. Keep the design simple
Volume	Terabytes per day	Partitioned columnar storage, parallel loads, cost controls
Sensitivity	Regulated personal data	Mask before load or enforce strict policies, plus audit trail
Availability	Report ready by 07:00	DAG SLA, retries, buffer time and lateness alerts
Change rate	Rules change weekly	Keep raw data and use SQL models so history can be rebuilt

Sizing: a worked estimate

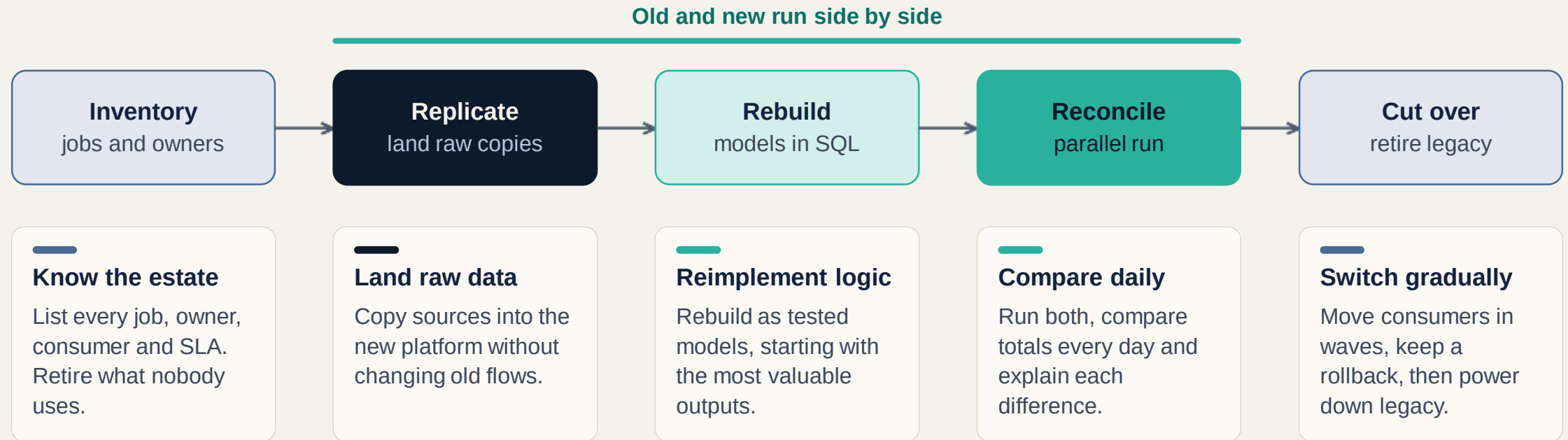
Quantity	Assumption	Result
Events per day	50 million	50 million
Average event size, JSON	1 KB	50 GB per day raw
Columnar compression	About 5 to 1, assumed	About 10 GB per day stored
Retention	13 months, about 395 days	About 3.9 TB stored
Peak ingest rate	3 times the average, assumed	About 1,700 events per second
Daily model scan	One partition plus a 3-day lookback	About 40 GB scanned

Every figure is an assumption for the exercise. Replace them with measured values, then add headroom for growth.

Anti-patterns and their fixes

Anti-pattern	Why it hurts	Better approach
Nightly full reload of huge tables	Time and cost grow with size	Incremental loads or CDC
Business logic inside the loader	Hard to test and reuse	Modelled, tested transformations
No raw copy	Cannot fix or replay history	Keep an immutable raw layer
now() inside jobs	Reruns give different results	Pass logical dates as parameters
Silent failures	Wrong data looks fine	Tests, monitors and alerts
One giant SQL script	Cannot debug or reuse	Small models in clear layers
Manual, undocumented steps	Fragile and unrepeatable	Everything as reviewed code

Migrating from legacy ETL to modern ELT



Migrate outputs, not jobs. Rebuild what consumers need instead of porting every legacy step.

A pipeline design review checklist

Design

- ✓ Purpose and consumers are named
- ✓ Freshness and SLA are written down
- ✓ The grain of each fact is stated
- ✓ Sensitive data handling is decided
- ✓ The pattern choice is justified
- ✓ A cost estimate exists

Operations

- ✓ Loads are idempotent and replayable
- ✓ Tests gate every published table
- ✓ Alerts have owners and runbooks
- ✓ The backfill procedure is documented
- ✓ Lineage and catalog entries exist
- ✓ Access follows least privilege

PART 13 · REFERENCE · REVIEW

Reference and review

Consolidate what you have learned: a glossary, one-page cheat sheets, a self-check, suggestions for further study and ten principles to remember.

[Glossary](#)[ETL vs ELT cheat sheet](#)[SQL patterns](#)[Self-check](#)[Next steps](#)[Ten principles](#)

Glossary (1 of 2)

Term	Meaning
Backfill	Re-running a pipeline over past periods to rebuild or correct data
CDC	Change data capture: streaming inserts, updates and deletes from a source log
Conformed dimension	A dimension shared across facts with identical keys and meaning
DAG	A directed acyclic graph of tasks and their dependencies
Data contract	A versioned agreement on schema, meaning and service levels
ELT	Extract, load, then transform inside the target platform
ETL	Extract, transform in a separate engine, then load
Fact table	A table of measurements at a stated grain
Grain	The meaning of one row in a fact table
Idempotent	Safe to run repeatedly, always leaving the same result

Glossary (2 of 2)

Term	Meaning
Lakehouse	Lake storage with warehouse features through open table formats
Lineage	The recorded path of data from source to consumer
Logical date	The data interval a run is responsible for, not the wall-clock time
Medallion	Bronze, silver and gold layers of increasing refinement
Orchestrator	A system that schedules, sequences, retries and monitors tasks
Reverse ETL	Syncing modelled warehouse data back into operational tools
Schema drift	Unannounced change to a source's columns or types
SCD	Slowly changing dimension: a way to handle attribute change over time
Surrogate key	A warehouse-owned integer that identifies a dimension row
Watermark	The high-water mark of what has already been extracted

Cheat sheet: ETL against ELT

Lean towards ETL when

- Data must be masked before landing
- The target cannot run heavy SQL
- Logic is non-SQL: parsing, ML, custom code
- Cost forces a small, curated target
- Regulation limits what may be stored

Lean towards ELT when

- The target is an elastic cloud platform
- Rules change and history must be rebuildable
- Analysts own the transformation logic
- Many sources must be onboarded quickly
- Raw data may serve later use cases

In both patterns: idempotent loads, tested models, orchestration with alerts, lineage and least-privilege access.

Cheat sheet: four SQL patterns

Latest row per key

```
select * from t
qualify row_number() over (
  partition by id order by ts desc) = 1;
```

Incremental filter

```
where updated_at >
  (select max(updated_at) from target)
```

Upsert

```
merge into t using s on t.id = s.id
when matched then update set ...
when not matched then insert ...;
```

Partition replace

```
delete from t where d = :ds;
insert into t
select ... where d = :ds;
```

Syntax varies by engine. The patterns do not.

Self-check: twelve questions

- 1 Define ETL and ELT in one sentence each.
- 2 Name three reasons to transform before loading.
- 3 Why does ELT keep a raw layer, and what does it enable?
- 4 State the grain of a sales fact table you design.
- 5 When is a Type 2 dimension the right choice?
- 6 How can a watermark miss late-committed rows?
- 7 Why is `now()` a bug inside a scheduled job?
- 8 How do you make a load idempotent?
- 9 What belongs in a data contract?
- 10 How would you detect a silent drop in row counts?
- 11 Sketch a hybrid ETLT flow for sensitive data.
- 12 Estimate storage for 50 million 1 KB events per day.

Answer aloud or on paper. If a question stalls you, revisit the matching part in the roadmap.

Where to go next



The Data Warehouse Toolkit

Kimball and Ross. The standard reference for dimensional modelling.



Fundamentals of Data Engineering

Reis and Housley. A lifecycle view of modern data engineering.



Designing Data-Intensive Applications

Kleppmann. Storage, streams, consistency and distributed systems.



Official documentation

dbt, Apache Airflow, Dagster and your warehouse vendor's guides, for hands-on practice.



Build a practice pipeline

Ingest a public API into a warehouse, model it in layers, then add tests and a schedule.



Read incident write-ups

Post-mortems from data teams teach failure modes faster than success stories.

Check for current editions and versions before buying or installing.

Ten principles to take with you

The short version of 140 slides.

- 1 Choose the pattern from constraints, not fashion.
- 2 Keep an immutable raw copy unless you cannot.
- 3 State the grain before you design a table.
- 4 Make every load idempotent and replayable.
- 5 Layer transformations and give each layer one job.
- 6 Test data like code, and gate what you publish.
- 7 Use logical dates, never the wall clock.
- 8 Model datasets as dependencies, not clocks.
- 9 Protect sensitive data by design, early.
- 10 Measure cost, freshness and quality continuously.